Which Strategy Wins at Rummikub, and Why? A Simulation Study of Heuristics, Reinforcement Learning, Search and an LLM Decision-Maker

Marcos de Pinho Tavares Proença

October 2026

Abstract

We study which simple strategy wins most often at two-player Rummikub and, more importantly, *why*. In a from-scratch simulator of the official rules (106 tiles, two jokers, 14-tile hands, 30-point initial meld, table rearrangement), a paired round-robin of seven interpretable strategies (3,360 games) is won by the *joker-saver*: it plays as many tiles as possible but spends a joker only if the move empties its hand (mean score 0.726 ± 0.027 against the field). Four ablations (13,000 games) isolate the cause: a soft joker penalty does not reproduce the effect, forbidding joker removal from the table does not remove it, it arises mostly after the opening meld, and forbidding the joker even as the final tile destroys it (0.189 ± 0.021). The saver wins by holding a wildcard until it can go out (71% of its wins end with a joker, against 4% for the control). That advantage is an artefact of counting games won. Official Rummikub is zero-sum in *tile points*, with a joker left in hand counting 30, and in 18,000 games scored both ways the saver is first in games won and sixth of ten in points, 5.6 [3.8, 7.4] points per game below max-tiles, because it ends its losses holding 0.80 jokers. Over sessions of ten hands with accumulated points, the format in which the game is actually played (3,600 sessions, 36,000 hands), the inversion is complete: the saver wins the most hands (5.4 of 10) and the fewest sessions (0.288 ± 0.030), losing 72.5% of sessions to max-tiles, and the session ranking is the exact reverse of the per-hand win ranking (Spearman -1.00). Releasing the joker before the end is first in both points and sessions. We then ask whether learning finds any of this. A tabular selector over the strategies, the Jev System One model (`jev-1.13.0`) as strategy selector or move chooser, a determinised Monte Carlo search and a small policy network trained by REINFORCE all fail to beat the best heuristics. Cloning the saver first and refining with group-relative advantages brings the network level with max-tiles and the saver (at best 0.550 ± 0.040 and 0.495 ± 0.040, exploratory); reward shaping, a wider league, a residual network, Monte Carlo returns, PPO (0.447 held out), a search that sees hidden hands, and rewarding the official score instead of the game outcome all add nothing. Training on larger tables helps from five players up and adds little at three and four. All claims are conditional on this simulator and strategy menu.

1 Introduction

Rummikub is a tile game in which players empty their hands by forming runs and groups on a shared table. Its *optimisation* subproblems are well studied: forming the best set of melds from a hand is polynomial-time solvable but becomes hard under generalisations [1], and integer linear programming solves full table-rearrangement problems [2]. Much less is known about which *playing*

policies win more often. Popular advice ("keep your jokers") is widespread but, to our knowledge, rarely tested. We ask three research questions.

RQ1 Among a menu of seven interpretable strategies, which wins most often — and does the answer survive the scoring metric printed in the rules and the multi-hand format in which the game is played?

RQ2 What *mechanism* explains the winner's advantage?

RQ3 Can reinforcement learning (RL), a language-model comparator or a small search agent, choosing a strategy or a move, do better than the best fixed one?

Status of the evidence. RQ1 is exploratory, with two exceptions: the metric comparison of Section 5.3 and the session experiment of Section 5.4 have development and test seeds and criteria fixed before running. The mechanism experiments for RQ2 were designed *after* seeing the RQ1 result, with four competing explanations and an informal prediction for each (Section 5.2); they were not formally pre-registered, so we treat them as confirmatory only in the sense of a corrected multiple-comparison family. All results hold for our simulator and our menu of strategies; they are not claims about optimal Rummikub play.

2 Related Work

Rummikub. The literature we found treats Rummikub as an optimisation problem: a polynomial-time algorithm for the one-hand puzzle [1], an integer-programming formulation for rearranging the table [2], a thesis counting the hands that can be melded in one turn [19], and a recent bachelor thesis that reduces the memory of the puzzle solver and classifies puzzle difficulty [4]. The last explicitly notes that the multi-player game (initial meld, table manipulation, strategy) has received little study and leaves strategy for future work. We found no study that evaluates learning or search agents on the full multi-player game against heuristic baselines; this is the result of a limited search (arXiv, AAAI/ICML proceedings and university thesis repositories; one earlier thesis was not accessible), not a proof of absence. The closest learning study we know is on a sibling game: Kotnik and Kalita compare temporal-difference learning with self-play against co-evolution of the value function in gin rummy, and co-evolution was superior [5].

Learning in large-action card games. DouZero reaches strong DouDizhu play by Monte Carlo returns, a network that scores encoded (state, action) pairs, and many parallel actors, which suits action sets that change from turn to turn [6]; our move network likewise scores concrete moves rather than a fixed action head. Suphx adds global reward prediction and oracle guiding (training with hidden information visible) to deep RL for mahjong [7]. Policy-gradient self-play with a pool of past checkpoints as opponents is common practice in tabletop-game RL [8], which our league runs (Section 5.8) follow in spirit. A recent large-scale study finds generic PPO [10] competitive with game-theoretic deep-RL methods in imperfect-information games [9]; we run a small PPO and a DouZero-style Monte Carlo variant (Section 5.13). Group-relative advantages, which replace a learned critic by standardising returns within a group of rollouts that share a prompt, were introduced for language-model training [14]; we use the same estimator with a group of games that share a deal. Our shaping term is a potential difference and therefore policy-invariant in the sense of [16]. ReBeL combines RL and search with convergence guarantees in two-player zero-sum games

[11]. The closest positive result to our own setup is IRumAI, which warm-starts PPO by behaviour cloning in Indian Rummy and reports a 53.9% win rate against the strongest search baseline unseen in training [18]; our recipe is the same in outline and does not reproduce that outcome (Section 6).

Search under hidden information. Perfect-information Monte Carlo sampling, which our PIMC agent follows, is a practical approach but suffers *strategy fusion*, since the search may choose different continuations in worlds the player cannot distinguish [17]; its empirical success has been analysed [12], and information-set MCTS searches over the player's information set instead of sampled worlds [13]. We use neither a learned value function nor a model of the opponent's hand in the search; Section 7 lists them as untested.

Evaluation practice. Our methodological conclusion is adjacent to work arguing that deep-RL comparisons rely too heavily on point estimates from few runs [15]. That critique concerns *statistical* reliability at a fixed metric; we report a case where the metric itself, not its variance, reverses the conclusion (Sections 5.3 and 5.4).

3 Simulator and Strategies

Game. Two players in the main experiments (Section 5.7 adds three and four); 106 tiles (4 colours $\times$ values 1–13 $\times$ 2 copies, plus 2 jokers); 14 tiles per hand. A turn either draws one tile or lays tiles down. The first lay-down (the *opening meld*) must be worth at least 30 points using only the player's own tiles and may not touch the table. After the opening a player may lay several melds, extend melds, and rearrange the whole table, provided every meld on the table is valid at the end of the turn and at least one tile from the hand was played. The two jokers are shuffled with the other tiles, so a player can hold them from the deal or draw them later. A game ends when a hand is empty; if the pool is empty and both players pass in succession, the game ends as well. Our main experiments score this case as a draw (0.5); Section 5.5 repeats the tournament under the official tie-break (fewest points left in hand, a joker counting 30) and under fewest tiles. Games are capped at 400 decisions (never reached). Seat 0 moves first.

Action space. At each decision the simulator lists *draw*, *lay down new melds*, and *rearrange* actions. Because exhaustive table search is expensive, rearrangements are a sample of at most 128 plans. This is the main fidelity limitation (Sections 5.6 and 7).

Strategies. Each chooses one of the legal actions.

Q-meta. A tabular Q-learner [3] whose actions are the seven strategies plus "draw" (it *selects* a strategy at each decision). State: opening status of both players, binned hand sizes, binned maximum playable tiles, pool size, whether a winning move exists, and whether the hand holds a joker. Reward is $+1/0/-1$ at the end only, $\gamma = 1$, ϵ decays linearly from 0.3 to 0.05. It trains against opponents drawn uniformly from the seven strategies and is evaluated greedily.

Table 1: The seven strategies.

random	uniform over legal actions, drawing included
max-tiles	play the move using the most hand tiles (ties: larger tile sum); draw if none
max-points	play the move with the largest tile sum
min-play	play the smallest legal move
joker-saver	like max-tiles, but consider only moves that use no joker, *unless* some move empties the hand; otherwise draw
no-rearrange	like max-tiles but never modify melds already on the table
cautious	play only if the best move uses ≥ 4 tiles or goes out; otherwise draw

Jev-meta. At each decision `jev-1.13.0` (TypeSafe System One; fixed version) receives a JSON description of the state and a multiple-choice question over the same menu, with each option's concrete consequence in the current position. If its confidence is below a pre-fixed 0.3, or the budget is exhausted, the agent falls back to max-tiles. Jev is a *comparator*: it is not trained by RL.

Jev-moves. A second Jev agent chooses the *concrete move*. The state lists the rules, the hand (sorted), the table sets, the hand's point total, the opponent's hand size and opening status, and the pool size; every legal action of the turn, including "draw", is one option of a multiple-choice question (median 30 options and maximum 177 over 1,114 sampled decisions, below the 255 allowed). Each option states the tiles played from the hand, how many tiles remain, the table sets removed and added, and whether it empties the hand; sums and counts are computed in code. Option order is shuffled with a fixed seed. The answer indexes an already validated action, so no illegal move can be played. There is no confidence threshold; only transport failures, invalid answers or an exhausted budget fall back to max-tiles. Single-option turns are not sent. One prompt was used and was not tuned on results.

PIMC. A search agent samples D worlds consistent with what it can see (the opponent's hand and the pool are reshuffled; only the opponent's hand *size* is used), scores at most $K = 5$ candidate moves (the joker-saver's move, the best by tiles, and "draw") by finishing the game with the joker-saver on both sides under common random numbers, and plays the best; ties go to the joker-saver's move. Rollouts do not rearrange the table, which is 7× faster; the real move list keeps rearrangements.

4 Experimental Protocol and Statistics

Pairing. For every seed, both seatings are played with the same deal; the unit of analysis is the per-seed mean score of the strategy of interest (win = 1, draw = 0.5, loss = 0). Pairing cancels the first-mover and deal effects: a strategy played against itself scores exactly 0.5 by construction (control row in Table 3). Seat 0 wins $50.8\% \pm 1.5\%$ of tournament games.

Intervals and tests. We report mean $\pm$ 1.96× the standard error across seeds. p-values are two-sided normal-approximation tests of mean = 0.5 over seeds ($n \geq 80$ seeds in all comparisons). In the mechanism family (13 groups, of which two are mirror controls with $p = 1$ by construction) p-values are corrected with the Holm step-down procedure [24]; tournament intervals are descriptive

and not corrected. For the pooled-field joker analysis (Section 5.9) the two hands of a game and the games of a seed are correlated, so the intervals there are percentile bootstrap intervals that resample seeds (1,000 replicates), not the normal approximation.

Seeds and counts. Tournament seeds 1000–1079, mechanism seeds 70000–70499, initial-joker seeds 50000 and up, Jev-meta seeds 2000–2099, Jev-moves seeds 4000–4069 (plus 4000–4003 in an 8-game pilot), PIMC and move-network evaluation seeds 100000 and up (network training seeds $\geq 5{\times}10^6$), RL evaluation seeds $\geq 9{\times}10^6$ (training seeds are disjoint). Pooled-field joker analysis: seeds 90000–91999 (4,000 games, 8,000 hands). Games: tournament 3,360 (21 pairs $\times$ 80 seeds $\times$ 2 seatings; replayed under two more tie-break rules); mechanism 13 groups $\times$ 500 seeds $\times$ 2 seatings (13,000); initial jokers 3 strategies $\times$ 2,000 seeds (6,000 mirror games); Q-meta 5 replicas $\times$ 1,200 training games and $5 \times 7 \times 20 \times 2$ evaluation games; Jev 600; Jev-moves 280 (70 seeds $\times$ 2 opponents $\times$ 2 seatings); move network 7,000 evaluation games (7 opponents $\times$ 500 seeds $\times$ 2 seatings) after 36,000 training games; PIMC 360 (60 seeds $\times$ 2 opponents $\times$ 2 seatings for $D = 4$, and 30 seeds for $D = 2$ and $D = 8$ against the joker-saver); Q-meta under the official rule 5 replicas $\times$ 1,200 training games, evaluated with the same protocol. Later experiments: rearrangement-cap profiles 150,000 and up, three- and four-player tables 120,000 and up (130,000 and up for retrained agents), the 12-strategy league 140,000 and up, cloning demonstrations 6×10^6 and up, exploratory diagnosis of the cloned networks 160,000 and up, confirmatory evaluation 170,000 and up (group-advantage and rearrangement variants 175,000 and up), the robustness matrix, shaping and wider-league runs 170,000 and up, tile-counting runs 210,000 and up, the official-points tournament on development seeds 220,000–220,119 and test seeds 225,000–225,199 (45 pairs $\times$ 200 seeds $\times$ 2 seats = 18,000 games), the official-reward networks on development seeds 230,000–230,099 (4,800 games; test seeds 235,000+ unspent), and the sessions on development seeds 240,000–240,079 and test seeds 245,000–245,119 (15 pairs $\times$ 120 sessions $\times$ 2 arrangements $\times$ 10 hands = 36,000 hands; session-length sensitivity 248,000+ and threshold diagnosis 249,000+). Training seeds 0–2 always start from the clone made with the same seed.

5 Results

5.1 RQ1: the heuristic tournament

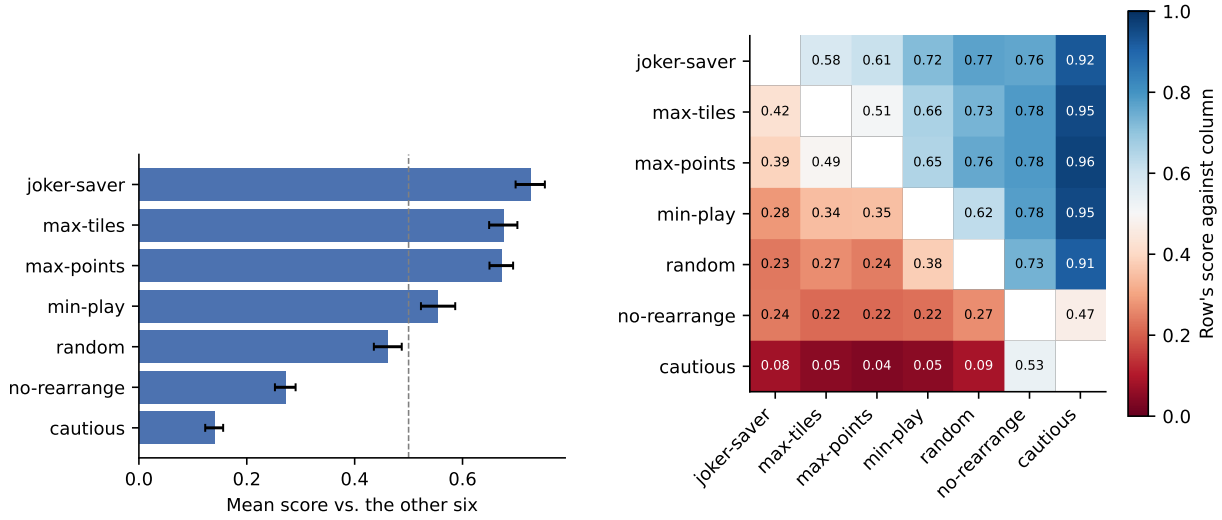
Table 2 and Figure 1a give each strategy's mean score against the other six. The joker-saver leads. Max-tiles and max-points are statistically indistinguishable (head-to-head 0.509 ± 0.059), and the joker-saver beats max-tiles by 0.578 ± 0.067 and max-points by 0.613; the full head-to-head matrix is in Figure 1b. Two designs are clearly bad: never touching the table (no-rearrange, 0.27) and drawing unless the move is large (cautious, 0.14). 16.3% of games end as pool-exhaustion draws (nearly all in games involving no-rearrange; Section 5.5), and games last 98 decisions on average. Section 5.3 asks whether this ranking is a property of the strategies or of the metric.

5.2 RQ2: why does the joker-saver win?

The joker-saver differs from max-tiles in a single respect, how it treats jokers, so the question is which part of that behaviour matters. We state four candidate explanations and what each predicts. All comparisons are against max-tiles as opponent.

Table 2: Round-robin score of each strategy against the other six (95% CI across seeds).

Strategy	Mean score
joker-saver	0.726 ± 0.027
max-tiles	0.676 ± 0.026
max-points	0.672 ± 0.022
min-play	0.555 ± 0.032
random	0.461 ± 0.026
no-rearrange	0.271 ± 0.019
cautious	0.140 ± 0.017



(a) Ranking.

(b) Head-to-head matrix.

Figure 1: Paired round-robin (3,360 games); score: win = 1, draw = 0.5.

E1: a mild preference for spending fewer jokers. Predicts that a soft penalty λ per joker in the move score (maximising $n_{\text{tiles}} - \lambda n_{\text{jokers}}$) reproduces the gain.

E2: recycling jokers on the table. Rearranging lets a player take a joker out of a meld; saving jokers might make them less exposed to such steals. Predicts that forbidding the removal of a joker from its table meld removes the gain.

E3: timing. Predicts that sparing jokers only before the opening, or only after, isolates where the gain arises.

E4: the joker as a finishing tile. A joker completes almost any set, so holding one may make going out possible. Predicts that forbidding the joker even for the last move removes (and reverses) the gain, and that most wins of the saver end with a joker.

E1 is not supported. Soft penalties of 0.5–4 tiles per joker give scores of 0.500–0.508, none distinguishable from 0.5 after correction, and a joker *bonus* is identical to the control. The reason is visible in Table 3: with soft penalties the player still spends 0.77 jokers per game, as often as the control (0.78); a preference only breaks ties between moves of equal size. The saver's behaviour

Table 3: Mechanism experiments: score of each variant against max-tiles (500 seeds × 2 seatings per row; the frozen max-tiles control is omitted), Holm-adjusted p for mean = 0.5, jokers played per game by the variant, jokers still in hand at the end of the game, and share of the variant's wins in which the final move used a joker. $\lambda > 0$ penalises jokers, $\lambda < 0$ rewards them.

Variant	Score	p_{Holm}	Jokers played	Jokers left	Win with joker
joker-saver (full rule)	0.568 ± 0.027	< 0.001	0.51	0.31	71%
joker-saver, jokers frozen on table	0.581 ± 0.027	< 0.001	0.53	0.32	75%
save only after the opening meld	0.537 ± 0.022	0.010	0.63	0.17	52%
save only before the opening meld	0.495 ± 0.021	1.000	0.77	0.01	5%
never play a joker, even to go out	0.189 ± 0.021	< 0.001	0.00	0.86	0%
soft penalty $\lambda = 4$	0.508 ± 0.014	1.000	0.77	0.01	6%
soft penalty $\lambda = 2$	0.506 ± 0.006	0.514	0.78	0.01	6%
soft penalty $\lambda = 1$	0.503 ± 0.003	0.661	0.78	0.01	4%
soft penalty $\lambda = 0.5$	0.500 ± 0.000	1.000	0.78	0.01	4%
joker bonus $\lambda = -1$	0.500 ± 0.000	1.000	0.78	0.01	4%
joker bonus $\lambda = -2$	0.500 ± 0.000	1.000	0.78	0.01	4%
max-tiles mirror (control)	0.500 ± 0.000	1.000	0.78	0.01	4%

is qualitatively different: it *refuses* to play joker moves and draws instead, keeping 0.31 jokers in hand at the end of an average game.

E2 is not supported. When jokers can never be removed from table melds, the saver's score is 0.581 ± 0.027, not lower than in the standard rules (0.568 ± 0.027). The saver does not owe its edge to joker recycling or table theft.

E3: the effect arises after the opening. Saving jokers only before the opening gives nothing (0.495 ± 0.021); saving only after it gives 0.537 ± 0.022 ($p_{\text{Holm}} = 0.01$). The two timings do not add up to the full rule (0.568), so part of the effect depends on both phases acting together; we have not isolated it.

E4 is supported. If the saver is forbidden to use a joker even to empty its hand, its score collapses to 0.189 ± 0.021 (Figure 2): it ends games holding 0.86 jokers on average and never wins by going out with one. Conversely, 71% of the full saver's wins end with a joker in the final move (against 4% for max-tiles). Saving jokers is therefore *only* beneficial when paired with spending them to go out. Because the full-rule and never-play variants differ *only* in that exception, this is a within-simulator causal test (an intervention on the policy), not just a correlation.

Explanation. The joker is a wildcard; spent early it saves a tile's worth of play, but held it keeps the hand completable. A hand that has been reduced to a few awkward tiles can often be finished only with a wildcard, so the saver trades a little early progress for the ability to finish when the opportunity arises. This is our interpretation of E3–E4; it is consistent with the interventions, but we did not test the narrower claim about "awkward last tiles".

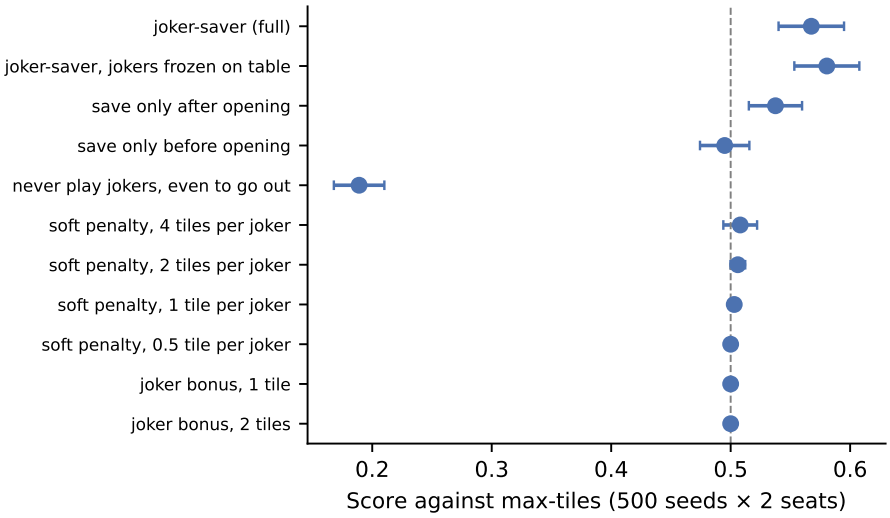


Figure 2: Mechanism experiments (mean ± 95% CI).

5.3 RQ1b: the official scoring metric

All results above score a game as win, draw or loss. Official Rummikub does not: the player who goes out scores the sum of the tiles left on every opponent's rack, each loser scores minus the value of its own rack, and a joker left in hand counts 30, so a deal is zero-sum in *tile points* [23]. Winning more games and scoring more points are different objectives, and the joker-saver is exactly the policy for which they might disagree: it ends its losses holding 0.80 jokers, which is 24 points of liability per lost game. We therefore replayed the tournament computing both metrics *from the same games*, so the two are perfectly paired, and added three strategies that decide when to let a joker go (defined before running, not tuned afterwards): *saver-then-dump* (the joker-saver, but once the pool is empty it plays the largest move even with a joker), *saver-dump-near* (the joker-saver, but it stops sparing jokers once an opponent is down to three tiles) and *joker-spender* (prefers moves that spend jokers; a control for the opposite extreme, since the joker-bonus variants of Table 3 only break ties and are therefore identical to max-tiles). The three criteria below were fixed before running, on development seeds (220,000+, 120 seeds); the test seeds (225,000+, 200 seeds × 2 seats, 18,000 games) were spent once, with Holm over the three. The ranking changes (Table 4, Figure 3). The joker-saver is first by win score (0.680 ± 0.017) and sixth of ten by official points ($+13.3 \pm 1.5$), below max-tiles and max-points ($+19.5$ and $+19.6$) which it beats in games won. Spearman's rank correlation between the two orderings is 0.64, against 0.96 between the end-of-game rules of Section 5.5: the choice of metric reorders the top far more than the choice of tie-break does. Against a common field of five strategies, paired by seed, max-tiles is +5.59 [+3.78, +7.39] points above the joker-saver while being -0.013 ± 0.018 below it in win score (criterion: $\geq +3.0$ with a positive interval; met, $p_{\text{Holm}} < 0.001$). Head-to-head the saver wins 0.573 ± 0.044 of the games and still loses 5.4 ± 3.1 points per game.

The cause is the mirror image of the mechanism of Section 5.2. Decomposing the saver's points against the common field: it wins 77.8% of those games, and when it wins it collects +48.0 points, the same as max-tiles (+47.9) — holding a joker buys the win, not a bigger win. When it loses it pays −43.3, against −14.6 for max-tiles, and 24 of those 29 extra points are the retained joker. The same rule that converts jokers into wins converts losses into expensive losses, and under the

Table 4: Both metrics from the same 18,000 games (test seeds 225,000+; 200 seeds, both seats; *: strategy added here). Each cell is the mean against the other nine strategies, ± 95% CI over seeds. Official points are per game and zero-sum, so the mean over a symmetric field would be 0 for all players; the spread reflects who beats whom. "Jokers left" is the mean number of jokers in hand at the end of a game.

Strategy	Win score	Official points	Points rank	Jokers left
saver-then-dump*	0.689 ± 0.017	$+15.0 \pm 1.4$	5	0.19
joker-saver	0.680 ± 0.017	$+13.3 \pm 1.5$	6	0.27
max-points	0.639 ± 0.019	$+19.6 \pm 1.0$	2	0.00
max-tiles	0.633 ± 0.015	$+19.5 \pm 1.0$	3	0.00
joker-spender*	0.633 ± 0.015	$+19.5 \pm 1.0$	4	0.00
saver-dump-near*	0.622 ± 0.021	$+19.8 \pm 1.1$	1	0.07
min-play	0.495 ± 0.025	$+12.6 \pm 1.3$	7	0.00
random	0.440 ± 0.020	$+9.7 \pm 1.1$	8	0.00
cautious	0.160 ± 0.013	-26.7 ± 1.8	9	0.01
no-rearrange	0.009 ± 0.004	-102.3 ± 1.9	10	0.01

official metric the second effect is the larger one.

Intervening on the policy confirms the joker is the channel, with a caveat. *Saver-then-dump* ends its losses with 0.64 jokers instead of 0.80 and gains +2.83 [+1.99, +3.67] points over the saver while *also* gaining +0.019 [+0.012, +0.025] in win score: dumping the joker once the pool is empty is free. The gain in points did not meet the +3.0 we had fixed, so we report it as a near miss rather than a confirmed effect; the no-regression criterion in win score was met. *Saver-dump-near* dumps much earlier, ends with 0.09 jokers and pays −17.7 per loss instead of −43.3: it is first of all ten in points ($+19.8 \pm 1.1$) and only sixth in win score (0.622 ± 0.021, -0.019 ± 0.014 against the saver). So the two objectives are reachable, by different policies, and the gap between them is governed by when the joker is released. *Joker-spender* is inert: under max-tiles play a joker is in hand in 2.6% of decisions and a choice between a joker move and a non-joker move of the same size arises in 0.1%, so it played identically to max-tiles in all 1,440 comparable games. This is a property of the action distribution, not a failure of the strategy.

We draw two conclusions. First, the answer to "which strategy wins" depends on what "wins" means, and the popular advice to save jokers is defensible only under the game-count metric: under the metric printed in the rules it costs about six points a game against simply playing the most tiles. Second, the finishing-joker mechanism of Section 5.2 is not overturned; it is priced. Holding a wildcard is a bet that pays a win and charges 30 points when it fails. All the reinforcement-learning experiments below optimise the ±1 game outcome, which we now know is the secondary metric; training on points is untested and is the first item of Section 7.

5.4 RQ1c: sessions of several hands

Rummikub is normally played as a *session*: several deals, points accumulated, highest total wins. Summing independent hands narrows the distribution around the mean, and the joker-saver's mean is negative, so the session should amplify what Section 5.3 found. We built a session engine: K hands, official points accumulated, the highest total wins, the starting seat alternating each hand as the turn to start passes in the rules (with K even this cancels the first-mover effect inside the session), and the seed of hand k derived from (session seed, k), so different strategies meet the *same* hands in the same position of the session. A session also creates information that no single-hand

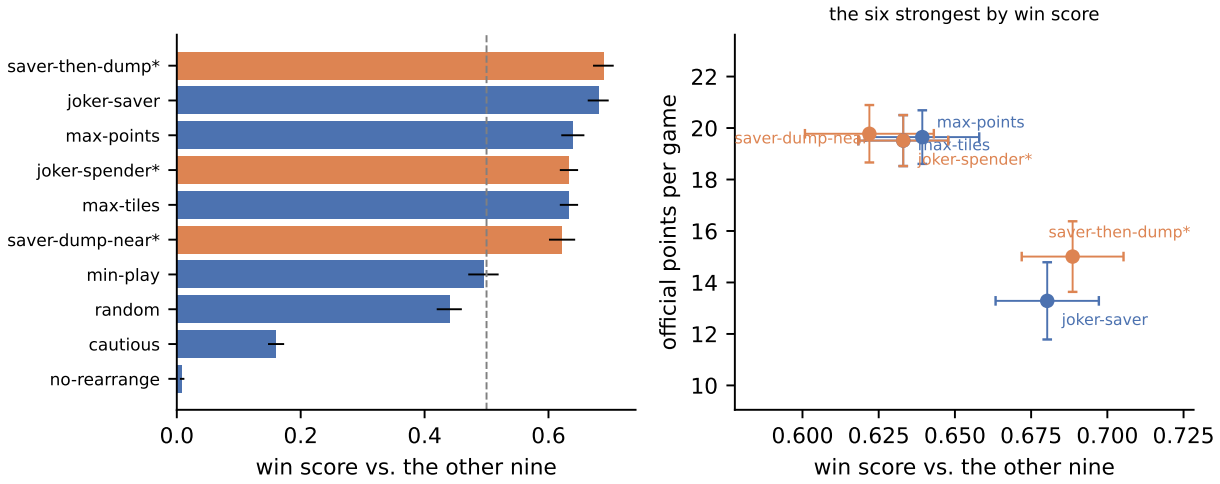


Figure 3: The two metrics disagree. Left: win score against the other nine (orange: strategies added in this section). Right: the six strongest by win score, in both metrics (± 95% CI); the joker-saver is first in win score and last of the six in points.

strategy has, the accumulated balance, so we added *session-adaptive*: when it is behind by more than one joker (−30 points, fixed in advance) it plays the joker-saver, maximising its chance of winning the hand; otherwise it plays saver-dump-near. Six strategies, 15 pairs, $K = 10$, both seat arrangements; development seeds 240,000+ (80 sessions per pair) and test seeds 245,000+ (120 sessions, 3,600 sessions, 36,000 hands), spent once, with Holm over the two primary criteria fixed before running. The inversion is complete (Table 5, Figure 4). The joker-saver wins 5.39 of the

Table 5: Sessions of $K = 10$ hands (test seeds 245,000+; 120 sessions per pair, both seat arrangements). Each cell is the mean against the other five, ± 95% CI over sessions. *: strategy not in the original seven.

Strategy	Sessions won	Points per session	Hands won of 10
saver-dump-near*	0.655 ± 0.028	$+47 \pm 5$	4.76 ± 0.11
session-adaptive*	0.621 ± 0.028	$+18 \pm 7$	4.96 ± 0.09
max-tiles	0.560 ± 0.039	$+27 \pm 9$	4.77 ± 0.15
max-points	0.548 ± 0.041	$+24 \pm 9$	4.67 ± 0.15
saver-then-dump*	0.329 ± 0.030	-49 ± 9	5.43 ± 0.11
joker-saver	0.288 ± 0.030	-67 ± 10	5.39 ± 0.11

10 hands in a session, more than any other strategy, and wins 0.288 ± 0.030 of the sessions, fewer than any other strategy. Head-to-head against max-tiles it takes 5.53 hands out of 10 and loses 72.5% of the sessions (0.725 [0.668, 0.782] for max-tiles, criterion ≥ 0.55 with the interval above 0.5; met, $p_{\text{Holm}} < 0.001$) at -75.6 ± 14.9 points per session. The ranking of sessions matches the per-hand *points* ranking of Section 5.3 (Spearman +0.90) and is the *exact reverse* of the per-hand win ranking (Spearman −1.00) on the five strategies the two experiments share. The effect grows with the session, as the arithmetic predicts: max-tiles beats the saver in 0.558 ± 0.089 of one-hand sessions, 0.754 ± 0.077 at $K = 5$ and 0.754 ± 0.077 at $K = 20$, with the points margin rising from +14 to +142.

Session-adaptive failed, and failed in the informative direction. It loses to saver-dump-near, the fixed strategy it switches away from (0.444 [0.414, 0.473]; criterion not met, and the deficit is itself

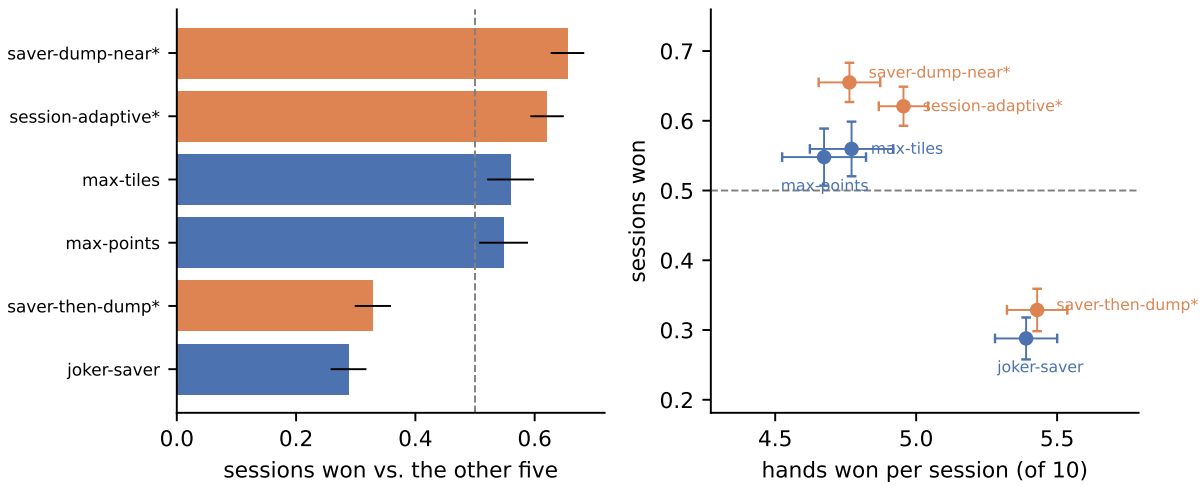


Figure 4: Winning hands and winning sessions pull apart. Left: sessions won against the other five (orange: strategies not in the original seven). Right: the same strategies by hands won per session and sessions won; the joker-saver wins the most hands and the fewest sessions.

significant, $p_{\text{Holm}} = 0.0002$). The rule is not inert: it is behind the -30 threshold in 26% of hands, so it does change posture often. Switching to joker-saver play when behind buys hands (4.96 per session, the most of the four strong strategies) and costs points, which is the same trade the whole section is about; chasing a deficit by saving jokers digs the hole deeper. The accumulated balance is real information that this rule turns into a loss, and we report it as a negative result rather than tuning the threshold, which was fixed in advance and tried at one value.

Two readings follow. First, the practical answer to the original question changes with the format: in a single hand scored by games, save your jokers; in a session scored by points, which is how the game is actually played, release them before the end — saver-dump-near is first in sessions won (0.655 ± 0.028) and the joker-saver is last. Second, this is a cautionary measurement about benchmark design rather than about Rummikub: a 0.573 per-hand win rate, the kind of number this literature routinely reports as an improvement, corresponds here to losing three sessions out of four. The quantity that is cheap to measure and the quantity the game rewards were anti-correlated on our menu.

5.5 Sensitivity to the end-of-game rule

When the pool is empty and both players pass, the official rule awards the game to the player whose remaining tiles are worth the fewest points (a joker counts 30). The main experiments score these games as draws (16.3% of the tournament). We replayed the same 3,360 tournament games (same seeds) under the official rule and under a second one, fewest tiles in hand, and replayed the two ablations that matter most for E4 (500 seeds $\times$ 2 seatings each). Table 6 gives the results; Spearman's rank correlation of the seven-strategy ranking with the draw-rule ranking is 0.96 for both alternatives. The top of the ranking and the mechanism do not depend on the rule. The joker-saver stays first and its lead grows ($0.726 \rightarrow 0.755$ and 0.765); max-tiles, max-points, min-play and random keep their order; head-to-head, the saver beats max-tiles by 0.561 ± 0.028 (official rule; 0.568 under the draw rule), and forbidding the joker even for the last move still collapses it (0.164 ± 0.021;

Table 6: Tournament score (95% CI) under three end-of-game rules. The last two rows are mechanism ablations against max-tiles. Fewest points is the official rule.

Strategy	Draw (0.5)	Fewest points	Fewest tiles
joker-saver	0.726 ± 0.027	0.755 ± 0.027	0.765 ± 0.027
max-tiles	0.676 ± 0.026	0.713 ± 0.025	0.713 ± 0.026
max-points	0.672 ± 0.022	0.710 ± 0.021	0.710 ± 0.021
min-play	0.555 ± 0.032	0.594 ± 0.030	0.594 ± 0.030
random	0.461 ± 0.026	0.509 ± 0.026	0.509 ± 0.026
cautious	0.140 ± 0.017	0.208 ± 0.018	0.208 ± 0.017
no-rearrange	0.271 ± 0.019	0.010 ± 0.006	0.001 ± 0.001
Draws (% of games)	16.3	0.0	0.1
Joker-saver vs. max-tiles	0.568 ± 0.027	0.561 ± 0.028	0.561 ± 0.028
Never-play-joker vs. max-tiles	0.189 ± 0.021	0.164 ± 0.021	0.166 ± 0.021

0.189 under the draw rule). Fewest points and fewest tiles are almost indistinguishable (0.0% and 0.1% of games remain tied), so the choice between them does not matter here. The only reordering is at the bottom: no-rearrange falls from 0.271 to 0.010 and below cautious. Its previous score was mostly pool-exhaustion draws: 54% of its games ended that way, against 1.1% of the other games, because a player that never touches the table seldom finishes. The official rule turns those stalemates into losses, so *the draw rule flattered the weakest strategy and accounts for most of the* 16.3% *draws*. The Jev-meta results were obtained under the draw rule and were not repeated; Q-meta was retrained and re-evaluated under the official rule (Section 5.12).

5.6 Sensitivity to the rearrangement search cap

The simulator lists rearrangements by a bounded search: at most 128 plans, 1,000 search visits, 2 table sets and 18 tiles per rearrangement. Instrumenting the search shows that the 128-plan cap is reached in 9.9% of decisions and in 73% of tournament games, so the cap is not harmless. We therefore replayed the tournament (7 strategies, 21 pairs, 40 seeds, both seats, official end-of-game rule) with larger caps: $\times 4$ (512 plans), $\times 16$ (2,048) and a large profile (4,096 plans, 3 sets, 24 tiles). Games are paired by pair, seed and seat. The no-rearrange / cautious pair was dropped from all columns because 47 of its 80 large-profile games had not finished when we stopped the run, so all columns use the same 1,600 games. The ranking is identical in the four columns (Table 7, Figure 5), and the joker-saver's advantage grows with the cap ($0.525 \to 0.556 \to 0.562 \to 0.625$ head-to-head over max-tiles), so truncation does not explain the result and, if anything, understates it. The head-to-head interval at the default cap includes 0.5 with only 40 seeds (the main tournament has more); the large-profile interval does not, but this comparison is exploratory. The collapse of no-rearrange in the large profile ($0.217 \to 0.045$) is consistent with opponents that rearrange finishing earlier (mean 99.5 vs. 111.9 decisions in its games); we did not test this further. An offline oracle re-enumerated 383 capped states with the large profile: it found a play with more tiles in 4.2% of them, always by exactly one tile (0.3% with $\times 4$). Jev, PIMC and the move network were not rerun with larger caps.

Table 7: Tournament score by rearrangement cap (plans per decision). The last row is joker-saver vs. max-tiles with a bootstrap 95% interval over 40 seeds.

Strategy	Default (128)	×4 (512)	×16 (2048)	Large (4096)
joker-saver	0.690	0.729	0.736	0.783
max-tiles	0.685	0.681	0.680	0.698
max-points	0.662	0.648	0.647	0.680
min-play	0.596	0.568	0.564	0.592
random	0.486	0.484	0.482	0.520
no-rearrange	0.217	0.220	0.219	0.045
cautious	0.039	0.048	0.050	0.028
Decisions at the plan cap (%)	9.9	1.6	0.1	0.4
Joker-saver vs. max-tiles	0.525 [0.42, 0.62]	0.556 [0.45, 0.66]	0.562 [0.46, 0.66]	0.625 [0.53, 0.72]

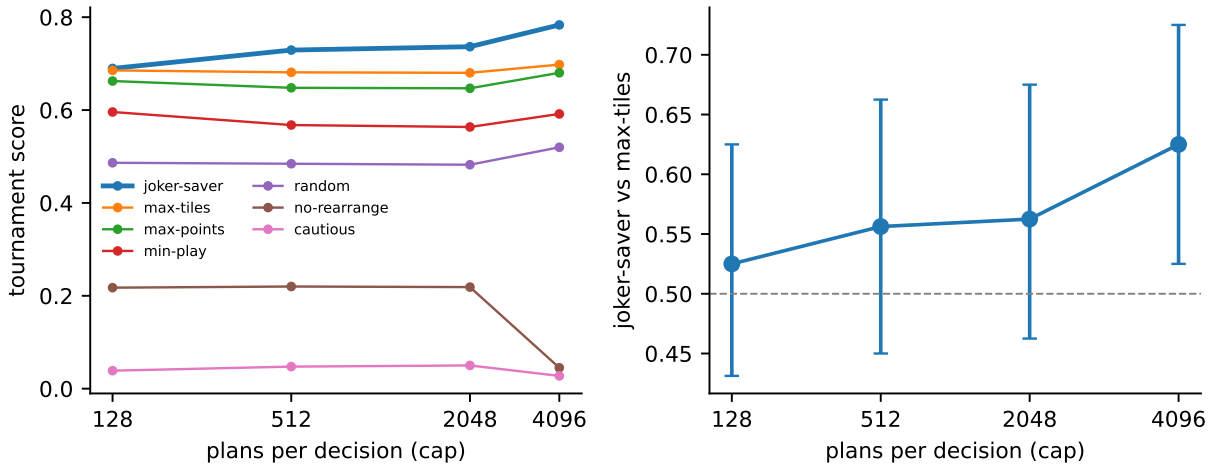


Figure 5: Left: tournament score of each strategy by rearrangement cap. Right: joker-saver against max-tiles with the 95% bootstrap interval over 40 seeds.

5.7 Beyond two players

The simulator accepts up to ten players; this section uses three and four (pools of 64 and 50 tiles after the deal; games last about 71 and 50 decisions against 96 for two players). The strategies of Table 1 do not look at opponents, so they run unchanged; the learned agents look at the opponents and were retrained for each table size (below); the Monte Carlo search was not rerun (Jev was, see the end of this subsection). One focus strategy plays all seats in rotation, so each seed gives N games, and we score a win as 1 and a draw as $1/N$, so the baseline is $1/N$. The opponents are either copies of max-tiles or a mixed field of $N-1$ distinct strategies drawn at random from the other six. Intervals are ± 1.96 standard errors over seeds. Each strategy has 100 seeds; the joker-saver against max-tiles has 500 (added after the first 100 were inconclusive, so this comparison is a follow-up rather than a pre-specified one). The lower half of the ranking is the same as with two players (Figure 6) (min-play, random, cautious and no-rearrange are far below chance). At the top, the picture changes. Against three max-tiles opponents at a four-player table the joker-saver scores 0.249 ± 0.015, exactly chance (0.25): its advantage disappears. At a three-player table it keeps a smaller advantage, 0.374 ± 0.021 against 0.333. The same strategy forbidden from playing a joker

Table 8: Score of one focus strategy at three- and four-player tables (win = 1; chance = $1/N$). "vs max-tiles": the other seats are max-tiles; max-tiles against itself is $1/N$ by symmetry. "Mixed field": other seats drawn at random among the six remaining strategies, so the field differs slightly between rows. The joker-saver's "vs max-tiles" cells use 500 seeds, all other cells 100. The two lower rows are learned agents trained for the table size: the network is the policy-gradient move network of Section 5.13 retrained at $N = 3$ and $N = 4$ (1,500 iterations; 100 evaluation seeds per cell), the Q-learner is the strategy selector retrained with five replicas (20 evaluation seeds each; intervals over replica–seed pairs). The no-winner row pools all rows.

	3 players ($1/N = 0.333$)		4 players ($1/N = 0.250$)	
Strategy	vs max-tiles	mixed field	vs max-tiles	mixed field
joker-saver	0.374 ± 0.021	0.567 ± 0.061	0.249 ± 0.015	0.378 ± 0.041
max-tiles	0.333 ± 0.000	0.536 ± 0.060	0.250 ± 0.000	0.421 ± 0.049
max-points	0.353 ± 0.038	0.470 ± 0.060	0.273 ± 0.027	0.416 ± 0.054
min-play	0.213 ± 0.039	0.412 ± 0.067	0.128 ± 0.031	0.246 ± 0.039
random	0.150 ± 0.038	0.267 ± 0.055	0.070 ± 0.026	0.139 ± 0.038
cautious	0.027 ± 0.018	0.063 ± 0.026	0.020 ± 0.013	0.024 ± 0.014
no-rearrange	0.008 ± 0.006	0.007 ± 0.005	0.003 ± 0.002	0.003 ± 0.003
network	0.390 ± 0.046	0.477 ± 0.057	0.275 ± 0.036	0.404 ± 0.044
Q-learner	0.130 ± 0.041	0.272 ± 0.052	0.112 ± 0.026	0.176 ± 0.040
Games with no winner (%)	0.2	0.6	0.2	0.8

even as its last move falls to 0.159 ± 0.015 at three players (500 seeds), as at two players (0.164), so the finishing-joker mechanism still holds where the advantage exists. In the mixed fields the top three strategies are not separable with 100 seeds. We do not know why the advantage shrinks at four players; a plausible but untested reason is that games end sooner, leaving fewer turns in which holding a joker can pay. Games with no winner are rare (0.1–0.7%). The learned agents behave differently from the heuristics. The tabular selector retrained at three and four players (five replicas, 1,200 training games each) ends *below* chance in every cell (Table 8; 0.130 ± 0.041 and 0.112 ± 0.026 against max-tiles): its coarse state summarises the opponents by one number and does not carry over. The move network retrained for each table size is the first learned agent that is not below its reference: against max-tiles it scores 0.390 ± 0.046 at three players and 0.275 ± 0.036 at four, statistically indistinguishable from the joker-saver (0.374 ± 0.021, 0.249 ± 0.015) at the same seeds, and only marginally above chance. In the mixed fields it is below the joker-saver at three players (0.477 ± 0.057 against 0.567 ± 0.061) and level at four (0.404 ± 0.044 against 0.378 ± 0.041). These are exploratory comparisons, with a single training run per table size and no multiplicity correction; we read them as "no worse than the best heuristic once the field is crowded", not as a win. A plausible but untested reading is that with more opponents the choice between drawing and playing matters less, which is also where the joker-saver's edge disappears. These are heuristic-versus-heuristic results, plus one training run of each learned agent.

Jev at three and four players. We ran the strategy-selecting Jev agent (same prompt, with the opponents summarised by hand size and whether they have melded) at both table sizes: 80 seeds, every seat, 1,680 games, US$1.43 in total. Against max-tiles opponents its result is identical to max-tiles in every game ($1/N$ exactly, 0.333 and 0.250). In a mixed field, on the same opponents as max-tiles and paired by seed, it scores 0.544 ± 0.068 and 0.398 ± 0.052, against 0.544 and 0.401 for max-tiles and 0.557 ± 0.056 and 0.426 ± 0.048 for the joker-saver (differences to max-tiles: 0.000 and -0.003 ± 0.006). The reason is visible in its choices: of 21,068 answers, 19,029 chose max-

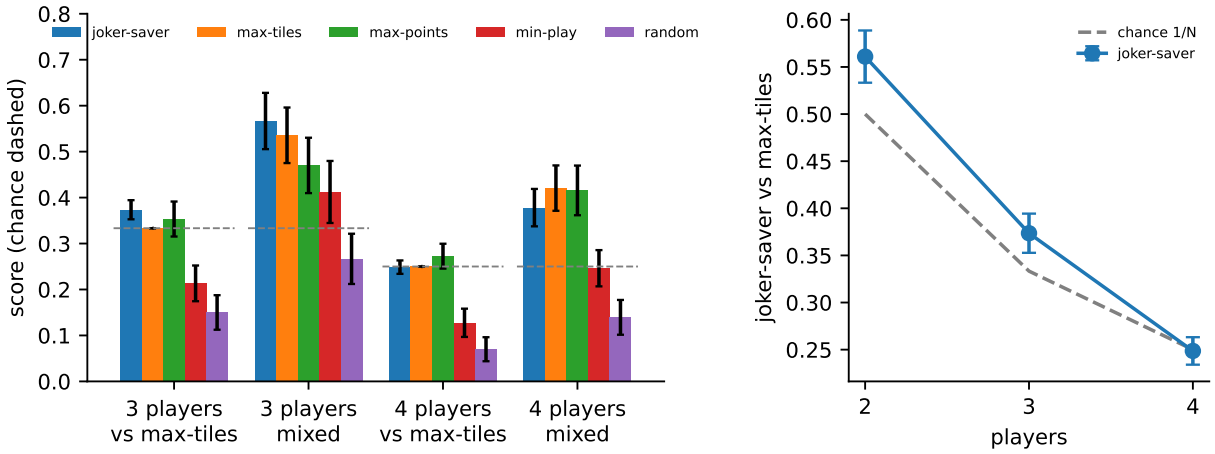


Figure 6: Left: score of five strategies at three- and four-player tables (dashed: chance $1/N$). Right: joker-saver against max-tiles as the number of players grows (± 1.96 standard errors over seeds).

tiles (including 9,884 low-confidence fallbacks to it), 2,039 chose "draw" (when the other options draw as well) and one chose another strategy; it never selected the joker-saver. With more players Jev still behaves as max-tiles. The agent that chooses the concrete move (Jev-moves) was also run at both sizes, on 40 seeds in the mixed field, paired with max-tiles on the same opponents (280 games, 3,251 calls, no fallbacks, US\$0.37): it scores 0.567 ± 0.108 against 0.589 ± 0.091 for max-tiles at three players (difference -0.022 ± 0.095) and 0.412 ± 0.080 against 0.408 ± 0.081 at four ($+0.005 \pm 0.086$), with the joker-saver at 0.603 ± 0.074 and 0.438 ± 0.076 on these seeds. It is indistinguishable from max-tiles at these table sizes, whereas against a single opponent it was clearly below it (0.350 ± 0.076); the sample is small and we do not claim the difference between table sizes is real. Total spend on the Jev agents at the larger tables: US\$1.80.

5.8 Stronger opponents: an extended league

The ranking above is relative to seven strategies we wrote. To see whether a more elaborate strategy displaces the joker-saver, we added five exploratory ones, defined before running and not tuned afterwards: *adaptive* (joker-saver until some opponent has ≤ 4 tiles, then max-tiles), *reorganiser* (prefers moves that avoid spending jokers and that move jokers already on the table), *flexible* (max-tiles, ties broken by the number of tile pairs left in hand that could still form a meld), and *saver-until-N* ($N = 2, 3$: the joker-saver, but a joker may be spent once the hand has N tiles or fewer). We ran a paired round robin of all 12 strategies (66 pairs, 80 new seeds, both seats, official end-of-game rule; 10,560 games, none truncated). Strength is also summarised by a Bradley–Terry fit, with a draw counted as half a win. The joker-saver stays first by mean score and Bradley–Terry strength (Table 9, Figure 7), but the three variants that keep a joker until the end (joker-saver, saver-until-2, saver-until-3) are not separable here: joker-saver against saver-until-2 is 0.519 ($p = 0.08$). Head-to-head against max-tiles the joker-saver scores 0.600 ± 0.075 ($p = 0.009$), but with 66 comparisons this does not survive the Holm correction, so the original result (more seeds, Section 4) remains the confirmatory one. Among the eight strongest strategies the only pairs significant after Holm involve the reorganiser, which loses to the joker-saver (0.369 ± 0.071) and to both saver-until variants: moving table jokers instead of playing more tiles does not pay. Adaptive (0.544 ± 0.068 against max-tiles) and flexible (0.500 ± 0.068) are not distinguishable from max-tiles

Table 9: Extended league (*: exploratory strategy). Mean score over the other eleven strategies, Bradley–Terry strength, and the share of the strategy's wins that ended with a joker, measured only in games where it occupies the first slot of the pair (saver-until-3 never does).

Strategy	Mean score	Bradley–Terry	Wins ending with a joker (%)
joker-saver	0.652	2.64	59
saver-until-2*	0.641	2.51	62
saver-until-3*	0.628	2.38	–
flexible*	0.624	2.33	8
adaptive*	0.620	2.29	18
max-points	0.602	2.11	5
max-tiles	0.593	2.03	4
reorganiser*	0.542	1.62	5
min-play	0.531	1.55	0
random	0.418	0.92	0
cautious	0.136	0.12	2
no-rearrange	0.013	0.01	0

or from the joker-saver. Of the 32 pairs that are significant after Holm, none forms a three-way cycle, so strength is transitive on this menu. The finishing-joker pattern reappears: 59–62% of the wins of the joker-saver and of saver-until-2 end with a joker, against 0–18% for the others. None of the new strategies is better than the joker-saver at this sample size; this does not show that none could be, nor does it say anything about human play.

5.9 Jokers in hand: initial deal and draws

How much is a joker in the initial hand worth? We played 2,000 *mirror* games (both seats use the same strategy; seeds 50000 and up) for random, max-tiles and joker-saver and grouped the 4,000 initial hands per strategy by the number of jokers they contain (Table 10, Figure 8). Each entry is the mean score of hands with that number of jokers; the two hands of a game are counted separately and treated as independent in the intervals, an approximation we did not check. A

Table 10: Mean score of an initial hand by number of jokers, mirror matches (95% CI; number of hands in parentheses).

Strategy (mirror)	0 jokers	1 joker	2 jokers
max-tiles	0.497±0.018 (3025)	0.507±0.033 (903)	0.542±0.116 (72)
joker-saver	0.466±0.018 (3025)	0.600±0.031 (903)	0.674±0.103 (72)
random	0.504±0.018 (3025)	0.494±0.033 (903)	0.403±0.114 (72)

joker in the opening hand is worth a substantial edge *only for the joker-saver*: with one joker it scores 0.600 ± 0.031 and with two 0.674 ± 0.103 (72 hands only, so the interval is wide), against 0.466 ± 0.018 with none. Under max-tiles the score is 0.497, 0.507 ± 0.033 and 0.542 ± 0.116, and under random 0.504, 0.494 ± 0.033 and 0.403 ± 0.114, so a joker confers no detectable advantage to a player that does not hold it for the final move. This agrees with Section 5.2 (E4): the value of a joker depends on the policy that uses it. The two-joker estimate is imprecise and should not be read as a precise win probability.

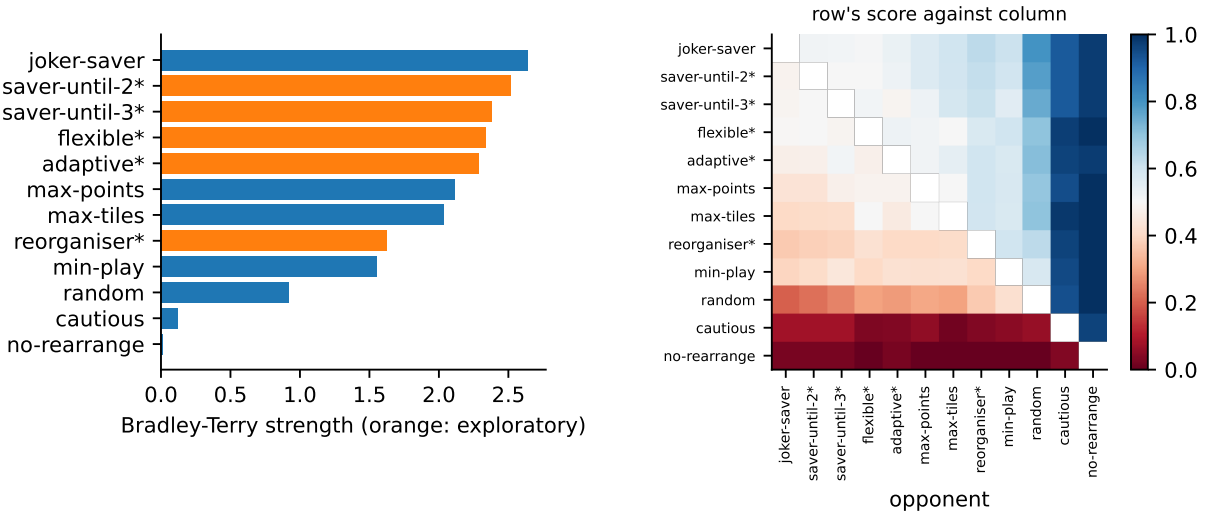


Figure 7: Extended league. Left: Bradley–Terry strength (orange: exploratory strategies). Right: score of the row strategy against the column strategy (80 seeds, both seats).



Figure 8: Score by number of initial jokers, mirror matches.

Pooled field. The mirror design covers only three strategies. To ask the question without fixing the strategy, we played 4,000 games (2,000 seeds × 2 seatings, seeds 90000 and up, official tie-break) in which each seed assigns one of the 28 pairings, with repetition, of the seven strategies, and pooled the 8,000 hands. Intervals resample seeds. Table 11 gives the result. A hand without an initial joker scores 0.496 ± 0.006, with one 0.509 ± 0.020, with two 0.576 ± 0.081 (118 hands): averaged over the field, a starting joker is worth about one or two points, and the interval for one joker contains 0.5. Comparing, within each strategy, hands with and without a starting joker gives $+0.025$ [0.000, 0.052].

Jokers drawn from the pool. A joker can also arrive by drawing. Each player chooses to draw 29.7 times per game on average (draws after the pool is empty are passes), so most of the 78-tile pool is drawn: 50% of hands (3,991 of 8,000) draw at least one joker (0.59 per hand). A naive comparison is misleading: hands that drew a joker score -0.015 [-0.043, 0.012] relative to those that did not, because players who draw more are usually behind and, drawing more, meet

more jokers. We therefore compare hands within strata of the number of draws (five groups cut at the quintiles 23, 27, 31, 38; the pooled estimate uses strategy × quintile cells, the per-strategy estimates use the quintiles within each strategy), averaging the stratum differences with weights $n_1 n_0/(n_1 + n_0)$. The adjusted difference is +0.037 [0.012, 0.064]. Figure 9 shows the estimate per strategy: only the joker-saver has a clearly positive effect (+0.119 [0.055, 0.186]), followed by max-points (+0.081 [0.017, 0.151]) and max-tiles (+0.053 [−0.015, 0.120]); random, cautious, min-play and no-rearrange are indistinguishable from zero. The joker-saver's drawn-joker effect agrees with its mirror-match initial-joker effect (+0.13). In the pooled field, however, the joker-saver's *initial*-joker difference is 0.003 (0.681 ± 0.066 with one or more, 0.678 ± 0.030 with none; 251 hands), about three standard errors below the mirror estimate; we did not find a reason, and note it as an open point (the opponent in the mirror also saves jokers for the last move, which may matter). Both

Table 11: Pooled field (seven strategies, 8,000 hands): mean score by number of initial and drawn jokers, and effect of having a joker (hands with ≥ 1 minus hands with none). Intervals: normal over seeds for the group means; bootstrap over seeds for the effects. Bracketed values are 95% intervals.

Group	Score (pooled field)	Hands
Initial jokers = 0	0.496±0.006	6068
Initial jokers = 1	0.509±0.020	1814
Initial jokers = 2	0.576±0.081	118
Bought jokers = 0	0.508±0.014	4009
Bought jokers = 1	0.498±0.015	3282
Bought jokers = 2	0.468±0.042	709
Initial joker, effect (strategy-stratified)	+0.025 [+0.000, +0.052]	
Bought joker, naive difference	−0.015 [-0.043, +0.011]	
Bought joker, stratified by draws	+0.037 [+0.012, +0.064]	

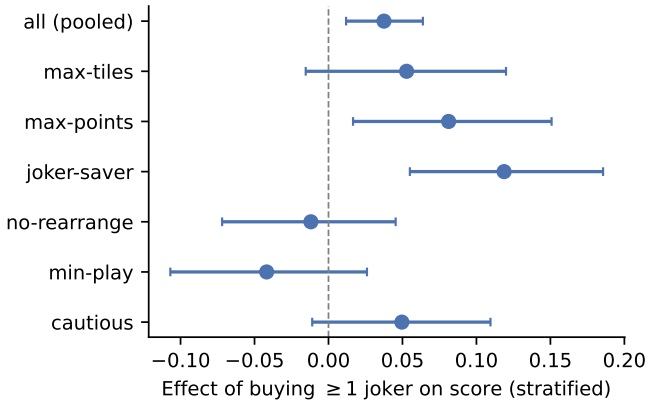
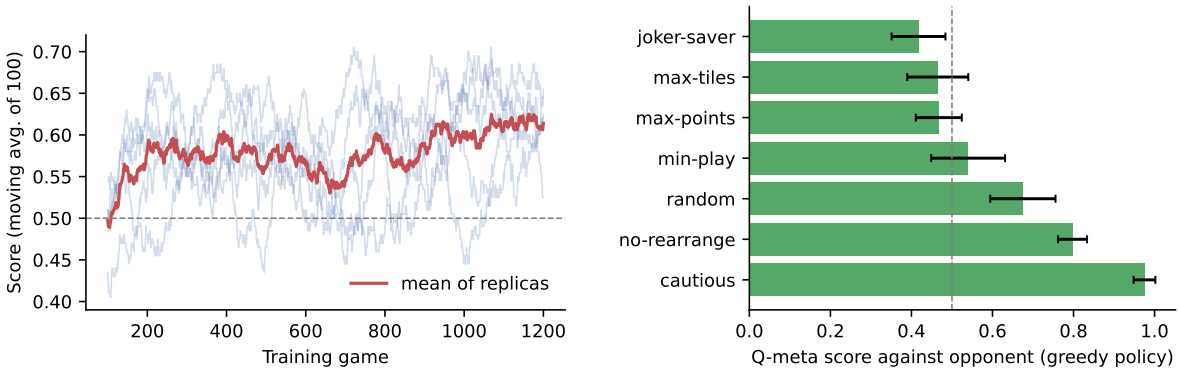


Figure 9: Effect of drawing at least one joker on the score, within strata of the number of draws, by strategy (95% bootstrap intervals).

routes lead to the same conclusion as Section 5.2: a joker is worth little to a player who spends it as soon as it helps, and its value is realised by the strategy that holds it for the last move. Because the player cannot choose when a joker arrives, this is a statement about the policy, not about the tile.

5.10 RQ3a: Q-learning over strategies

The moving-average training score rises from 0.50 (first 100 games) to 0.61 (last 100), averaged over five replicas (Figure 10a). Against the field the greedy policy scores 0.62 ± 0.05, but this is dominated by weak opponents (Figure 10b): against the best heuristics it scores 0.42 ± 0.07 (joker-saver), 0.47 (max-tiles) and 0.47 (max-points), with intervals including 0.5 for the latter two, and against random 0.675 ± 0.081, a point estimate below that of the fixed max-tiles (0.731) and joker-saver (0.766) against random, though the intervals overlap. The learned policy therefore does not exceed the best fixed strategy. This is consistent with the coarse state, a sparse terminal reward and only 1,200 episodes per replica; we cannot say whether more training or a richer state would change the conclusion. The learned policies are diffuse (Figure 11), so we make no claim about when to switch strategy.



(a) Learning curve.

(b) Q-meta against each opponent.

Figure 10: Q-meta training and evaluation (5 replicas).

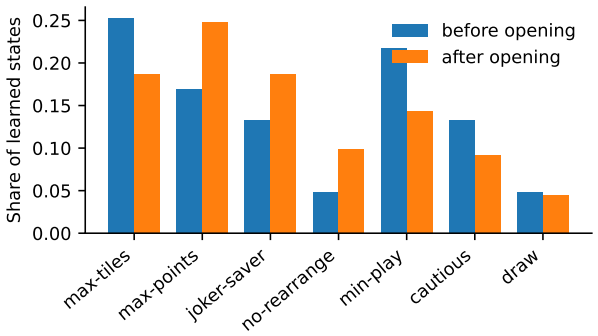


Figure 11: Share of learned states where each strategy has the highest Q-value, by opening status.

5.11 RQ3b: Jev as a strategy selector

Over 600 games Jev made 28,090 calls (29.2M input tokens, US$1.23 at US$0.042 per million tokens; output is free; 7 transport failures). In 49% of calls confidence fell below 0.3 and the fallback (max-tiles) was used; of the 14,237 answered decisions, 84% chose max-tiles and 16% "draw"; the other

options were essentially never chosen (Figure 12). It scores 0.50 against max-tiles, 0.68 ± 0.06 against random and 0.40 ± 0.06 against joker-saver: in this protocol it behaves like max-tiles and misses the joker-saver's mechanism (E4), which its prompt did not describe. The sample is small and exploratory, with one prompt and one model version.

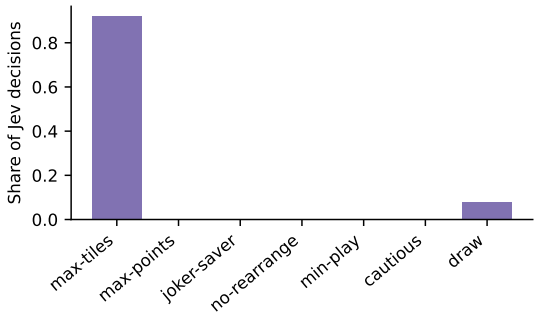


Figure 12: Distribution of Jev's choices (including fallbacks).

5.12 Q-meta under the official end-of-game rule

We retrained the five Q-meta replicas (1,200 games each) and re-evaluated them with fewest points deciding a stalled pool. Scores (100 seed pairs per opponent, pooled over replicas; the interval ignores clustering by replica and is therefore optimistic) are below; the draw-rule values from the previous subsection are in brackets.

Opponent	Official rule	[Draw rule]
joker-saver	0.370 ± 0.060	$[0.417 \pm 0.072]$
max-tiles	0.410 ± 0.066	$[0.465 \pm 0.063]$
random	0.595 ± 0.062	$[0.675 \pm 0.058]$
mean over the seven opponents	0.616 ± 0.027	$[0.620 \pm 0.026]$

The conclusion is unchanged: the learned selector remains below the joker-saver and max-tiles, and the field average is dominated by weak opponents. Part of the draw-rule score against the strong heuristics came from stalled-pool draws.

5.13 RQ3c: agents that choose the move

The three agents below were evaluated under the official rule, paired by seed with alternating seats.

Jev-moves. Over 280 games Jev answered about 6,000 decisions (about 2,000 input tokens per call, US$0.0018 per game, US$0.51 in total); the fallback was used 7 times. Its mean confidence was 0.38, low as expected with about 30 options. Its choice coincided with max-tiles' choice in 40% of decisions, it drew in 3% and played a joker in only 3–4%, so it neither reproduces the biggest-move rule nor the finishing-joker rule. It is clearly worse than max-tiles and worse than the joker-saver. The prompt gave the rules and the consequence of every option but no strategy advice; other prompts (for instance one that stresses emptying the hand) were not tried.

Table 12: Move-choosing agents against fixed heuristics (score ± 95% CI over seeds; p for mean = 0.5, uncorrected).

Agent	Opponent	Seeds	Score	p
Jev-moves	max-tiles	70	0.350 ± 0.076	< 0.001
Jev-moves	joker-saver	70	0.414 ± 0.072	0.019
PIMC D=4	max-tiles	60	0.542 ± 0.091	0.37
PIMC D=4	joker-saver	60	0.433 ± 0.085	0.13
PIMC D=2	joker-saver	30	0.267 ± 0.122	< 0.001
PIMC D=8	joker-saver	30	0.400 ± 0.128	0.13
Move network	joker-saver	500	0.403 ± 0.030	< 0.001
Move network	max-tiles	500	0.493 ± 0.029	0.63
Move network	max-points	500	0.492 ± 0.029	0.59
Move network	min-play	500	0.634 ± 0.029	< 0.001
Move network	random	500	0.732 ± 0.027	< 0.001

PIMC. With $D = 4$ worlds the search is statistically indistinguishable from max-tiles (0.542) and from the joker-saver (0.433, whose interval includes 0.5), but with 60 seeds the power is low. More worlds did not help ($D = 8$: 0.400) and fewer clearly hurt ($D = 2$: 0.267). The cost is about 105 s per game for $D = 4$ against about 1.3 s for a heuristic. Because the rollout policy is itself the joker-saver, the search can at best correct its choices at the margin, and ties are resolved in its favour; we therefore read this as "search with this rollout does not beat the joker-saver", not as a bound on search in general.

Move network. To remove the strategy menu from the learner we trained a small policy that scores each legal move directly: a network with 15 features per move (move size, jokers used, hand and pool sizes, opening status, ...), one hidden layer of 16 tanh units and a softmax over the legal moves, written in plain Python. It was trained by REINFORCE with an exponential-moving-average baseline, Adam (3×10^{-3}) and an entropy bonus (0.01) for 1,500 iterations of 24 games (36,000 games), with reward ±1 at the end of the game. Opponents were drawn from a league: 50% the seven heuristics, 25% the current network, 25% one of its last ten snapshots. It is evaluated greedily on 500 held-out seeds per opponent, both seatings (7,000 games, none truncated). It plays like max-tiles: 0.493 and 0.492 against max-tiles and max-points (both intervals include 0.5), where max-tiles itself scores 0.506 against max-points. It beats the weaker heuristics by about as much as max-tiles does (0.732 against random, where max-tiles scores 0.731; 0.634 against min-play, 0.656). Against the joker-saver it scores 0.403 ± 0.030, below 0.5 and close to what max-tiles obtains (0.425 in the tournament, 80 seeds). The finishing-joker rule was not learned: it went out with a joker in 2.9% of games (max-tiles 3.5%, joker-saver 41%) and played 0.80 jokers per game (max-tiles 0.83). The training-time win rate against heuristics rose slowly from 0.59 to 0.64 (blocks of 250 iterations); it is measured on a changing league and is not evidence of strength. This is one run of one architecture and one learning rate, with no hyper-parameter search, so we read it as "this learner, with this budget, converges to max-tiles-like play" and not as a limit of reinforcement learning.

Starting from imitation, and eighteen variants (exploratory). Because the network trained from scratch never found the finishing-joker rule, we first asked whether the features can represent it at all. We cloned the joker-saver by supervised learning on 10,587 of its decisions (300

Table 13: Eighteen variants started from the clone of the joker-saver, scored by game count (± 95% CI; "Games" pools the training seeds). Rows with a signed value are paired differences against the same architecture without the change, on identical deals. Every row is exploratory except the PPO held-out row; p-values are not corrected across rows. Seed ranges in Section 4.

Agent	Variant	vs max-tiles	vs joker-saver	Games	Verdict
Network	clone of the saver, untrained	0.525 ± 0.069	0.455 ± 0.069	200	below the saver
Network	clone + REINFORCE, seed 0 only (first look)	0.605 ± 0.07	0.545 ± 0.07	160	did not replicate
Network	clone + REINFORCE, 3 seeds pooled	0.509 ± 0.045	0.487 ± 0.045	480	level with both
Network	clone + moving-average baseline, 3 seeds	0.520 ± 0.040	0.480 ± 0.040	600	level with both
Network	clone + group advantage K=4, 3 seeds	$\mathbf{0.550 \pm 0.040}$	0.495 ± 0.040	600	level, $p = 0.014$ uncorr.
Network	+ hand-size shaping (0.5), 3 seeds	0.505 ± 0.039	0.468 ± 0.041	600	no gain
Network	+ nine-opponent training league, 3 seeds	0.546 ± 0.061	0.500 ± 0.064	240	no gain
Network	+ rearrangement during training, 1 seed	0.535 ± 0.069	0.510 ± 0.069	200	no gain
Network	+ rearrangement cap ×4 at evaluation	0.579 ± 0.056	0.463 ± 0.057	240	no clear change
Network	residual over the saver, $\beta \in \{1, 1.5, 4\}$	0.455–0.465	0.510–0.520	200	never departs
Network	Deep Monte Carlo [6], 3 seeds	0.555	0.472 ± 0.068	600	below the saver
Network	PPO [10], 3 seeds, development	0.568 ± 0.040	0.500 ± 0.040	600	level, not better
Network	PPO, held-out seeds, spent once	0.560 ± 0.056	0.447 ± 0.056	900	did not replicate
Search	determinised, *true* world revealed	0.508 ± 0.089	0.508 ± 0.089	120	below the saver
Network	64 hidden units (vs 16, same regime)	-0.088 ± 0.048	–	200	worse
Network	256 hidden units (vs 16, same regime)	$+0.012 \pm 0.047$	–	200	no gain
Network	19 opponent features (vs 15)	-0.010 ± 0.021	–	1440	no gain
Network	23 tile-counting features (vs 19)	-0.007 ± 0.037	–	720	no gain

games against the seven heuristics, with rearrangement; the same likelihood as REINFORCE with advantage 1). The clone agrees with the saver's choice in 82% of held-out decisions and goes out with a joker in 70% of its wins against the saver (71% for the saver itself). So the 15 features and one hidden layer suffice to represent the mechanism; what the from-scratch run lacked was the search for it. Cloning agreement stays at 0.82 for every width (16, 64, 256 units) and every feature set (15, 19, 23), so capacity is not what caps imitation either.

From those clones we ran the eighteen variants of Table 13, each with the protocol and the success criterion fixed in writing before running: three training seeds unless noted, 300 iterations of 24 games, learning rate 3×10^{-3}, greedy evaluation on fresh seeds, paired by deal and seat. The table is the result; three things in it are worth more than the individual rows.

First, one positive estimate did not replicate, and we report the failure. On the first training seed the cloned-then-refined network scored 0.605 ± 0.07 against max-tiles and 0.545 ± 0.07 against the saver, the only result in this project that appeared to beat both. Two further training seeds on fresh seeds gave 0.509 ± 0.045 and 0.487 ± 0.045: the first number was a favourable draw of seeds and run, selected by looking. PPO repeated the pattern at larger scale, reaching 0.500 against the saver on development seeds and 0.447 on held-out seeds spent once. Both are the expected behaviour when the best of several candidates is chosen on the data used to choose it.

Second, group-relative advantages [14] are the only change that did anything, and it is small. With four games sharing each deal and the advantage standardised within the group, the agent scores 0.550 ± 0.040 against max-tiles ($p = 0.014$ uncorrected), the only interval in this paper excluding 0.5 against max-tiles. We read it cautiously: it is one of four comparisons, and its difference from the same clones trained with a moving-average baseline on the same seeds (0.520 ± 0.040) is not itself significant. The more consistent effect is on behaviour: with grouping the agent goes out with

a joker in 59–66% of its wins in all three trainings, against 14–43% without it, so the lower-variance signal preserves what the clone knew rather than teaching something new.

Third, the obvious explanations for the ceiling are each ruled out. Hidden information is not binding: a determinised search given the *true* opponent hand and draw order, rather than sampled ones, scores 0.508 ± 0.089 against the saver. Capacity is not binding: 64 units is clearly worse (-0.088) and 256 is no better ($+0.012$), and the 64-unit loss is not unstable training (its training win rate rises like the others) but overfitting to the training league, visible only against an external evaluator. The state is not binding either: nineteen opponent features and twenty-three with tile counting leave the result unchanged under the curriculum regime. Opponent diversity is not binding: a nine-strategy training league moves nothing (0.546 against max-tiles, against 0.517 before widening). And a residual network that starts *as* the saver, adding its score to a bonus on the saver's move, never departs from it — in none of 270 greedy decisions per seed at $\beta \in \{1.5, 4\}$, and in 7 of 225 with $\beta = 1$ and 6.7 times the games — so the policy-gradient signal at this budget cannot even find a local improvement on the heuristic it starts from.

Robustness to opponents never seen in training (exploratory). The group-advantage networks and, for contrast, the moving-average ones were played against all twelve strategies of Section 5.8 (40 paired seeds, both seats, three training seeds pooled; Table 14). Five of the twelve were never seen in training. The "drop" between seen and unseen opponents cannot be read directly, because the unseen set is harder for everyone — the joker-saver itself falls from 0.740 to 0.535 — so we compare each agent with the heuristic references on the same opponents. On unseen opponents the group-advantage agent scores 0.532, the same as max-tiles (0.532) and the joker-saver (0.535); on seen opponents its mean (0.693) is 0.03–0.05 below theirs. Nothing breaks and nothing improves: the network behaves like max-tiles against everyone.

Table 14: Score against each opponent ($\pm$ 95% CI over seeds; $*$: not seen in training). "Group K=4" and "Moving avg." pool three training seeds. "Mean" averages the cells shown; the max-tiles and joker-saver columns lack their own diagonal. The worst case is the minimum cell and is biased downward.

Opponent	Group K=4	Moving avg.	max-tiles	joker-saver
random	0.783 ± 0.055	0.746 ± 0.056	0.662 ± 0.089	0.775 ± 0.086
max-tiles	0.517 ± 0.062	0.483 ± 0.062	–	0.500 ± 0.099
max-points	0.517 ± 0.060	0.517 ± 0.060	0.500 ± 0.078	0.487 ± 0.102
min-play	0.717 ± 0.054	0.738 ± 0.051	0.738 ± 0.099	0.750 ± 0.099
joker-saver	0.471 ± 0.062	0.479 ± 0.064	0.500 ± 0.099	–
no-rearrange	0.963 ± 0.023	0.992 ± 0.012	1.000 ± 0.000	0.975 ± 0.034
cautious	0.887 ± 0.041	0.921 ± 0.033	0.950 ± 0.047	0.950 ± 0.059
adaptive*	0.521 ± 0.060	0.492 ± 0.064	0.600 ± 0.106	0.575 ± 0.090
reorganiser*	0.608 ± 0.058	0.633 ± 0.061	0.525 ± 0.099	0.475 ± 0.116
flexible*	0.571 ± 0.067	0.504 ± 0.064	0.512 ± 0.089	0.600 ± 0.117
saver-until-2*	0.471 ± 0.062	0.487 ± 0.064	0.500 ± 0.105	0.500 ± 0.000
saver-until-3*	0.487 ± 0.061	0.496 ± 0.065	0.525 ± 0.116	0.525 ± 0.060
Mean, seen (7)	0.693	0.696	0.725	0.740
Mean, unseen (5)	0.532	0.522	0.532	0.535
Worst case	0.471 (joker-saver)	0.479 (joker-saver)	0.500 (max-points)	0.475 (reorganiser*)

Deep Monte Carlo and PPO (confirmatory for PPO). Two variants transfer techniques from other games to our network, both from the clone, with a league of heuristics, the current network and past checkpoints as opponents, 300 iterations of 24 games, learning rate 10^{-3}, four gradient steps per iteration and three training seeds each, fixed before running. Deep Monte Carlo [6] regresses the score of each played move to the final return; its value loss ends near 1.0, the variance of the returns, so the regression never learned to predict them, and it stays below the saver (0.472 ± 0.068). PPO [10] uses a separate value network on a seven-feature state, a normalised advantage, ratio clipping at 0.2 and a KL penalty (0.1) towards the clone. On development seeds it was the only candidate on par with both heuristics (lower bound ≥ 0.45), so we spent the held-out seeds on it once (150 seeds, both seats, three training seeds, 900 games per opponent; intervals over the 300 deals, which the training seeds share). Against the saver it scores 0.447 [0.391, 0.503], *not* on par, so the development result did not replicate. Against max-tiles it scores 0.560 [0.504, 0.616], above 0.5 ($p = 0.018$ one-sided) but not after correcting for the ten hypotheses of this family ($p = 0.18$). No variant tried beats the joker-saver.

Stress training on tables of up to ten players. The network trained for two players loses ground at tables of three and four, so we extended the engine to two to ten players (one set of 106 tiles per four players, 14 tiles each; ten players use 318 tiles and six jokers) and trained with a curriculum: each training game draws its table size uniformly from $2, \ldots, 10$. Everything else repeats the group-relative recipe ($K = 4$, 300 iterations of 24 games, learning rate 3×10^{-3}, start from the clone, three training seeds), so the control is the same network trained at two players only. The protocol was fixed before training, and the plan's five variants were already used, so this counts as a separate hypothesis. Table 15 shows the development seeds (176,000+; intervals over deals, paired with the control; tables of six and ten use only 20 seeds). The curriculum helps at larger tables, by +0.040 at three players (interval includes zero), +0.066 at four, +0.050 at six and +0.090 at ten, in the mixed field, and ends above the joker-saver at six and ten players. It costs at two players: 0.430 against the saver [0.367, 0.493], not on par by the margin set in advance, and 0.493 against max-tiles. Our pre-set criterion required a gain of at least 0.03 with an interval above zero at both three and four players, and no loss at two; it was not met (three players fails, two players fails), so the held-out seeds were left untouched. The pattern, however, suggested a new hypothesis, which we stated before touching them: the curriculum beats the control in the mixed field at four, six and ten players (Holm over the three tests; 40 to 60 seeds, both seats at every seat, networks unchanged), with the loss at two players as a descriptive check. On held-out seeds (190,000+, used once) it holds: +0.049 [0.008, 0.091] at four players, +0.038 [0.006, 0.069] at six and +0.060 [0.036, 0.084] at ten (adjusted $p = 0.042$, 0.042 and < 0.001); at three players, descriptively, +0.062 [0.010, 0.114]. At two players the curriculum is again lower, 0.427 against the saver [0.364, 0.489], −0.051 against the control (interval includes zero). Against the saver itself the curriculum is ahead at six and ten players (+0.074, +0.063) and level at three and four. The effects are small (0.04 to 0.06 points) and, at four players, smaller than on the development seeds (0.066). The treated network saw large tables in training and the control never did, so the gain is in-distribution rather than a sign of transfer; training up to six players and testing at ten would be the test for that, and we did not run it.

Training only on harder tables. The curriculum included the tables we measured, so we asked the question the other way round: train only at six to ten players (never two to five) and test at three and four. The recipe is the same (three training seeds), the criterion was set before training, and the development result did not meet it (gain +0.039 at three players with the interval

Table 15: Stress training (tables of 2–10 players) against the two-player control. Points in the mixed field ($1/N$ is a symmetric tie); difference with 95% interval over deals. Development seeds (176,000+; 20 seeds at $N = 6, 10$, otherwise 60) and held-out test seeds (190,000+; 40 seeds at $N = 6, 10$, otherwise 60).

Table	Development: curriculum − control	Test: curriculum − control
$N = 2$ vs saver	−0.043 [−0.103, 0.016]	−0.051 [−0.109, 0.007]
$N = 3$	+0.040 [−0.006, 0.085]	+0.062 [0.010, 0.114]
$N = 4$	+0.066 [0.020, 0.111]	+0.049 [0.008, 0.091]
$N = 6$	+0.050 [0.001, 0.099]	+0.038 [0.006, 0.069]
$N = 10$	+0.090 [0.049, 0.131]	+0.060 [0.036, 0.084]

including zero, +0.061 at four). We then tested, once and on held-out seeds (195,000+), the post hoc hypothesis that this network beats the two-player control at three and four players; it did not. The difference in the mixed field is +0.035 [−0.020, 0.090] at three and −0.002 [−0.046, 0.041] at four (Holm-adjusted $p = 0.42$ and 0.91). At five players, never seen in training and not part of the criterion, the gain is +0.095 [0.047, 0.143] (development: +0.060). The network trained only on hard tables is indistinguishable from the curriculum network at every table size (differences of at most 0.01 with narrow intervals), so what matters is having seen large tables, not the ones measured. The same seed set also weakens the earlier curriculum result: at four players the curriculum scores −0.004 against the control (development +0.066, first test +0.049), and the control's cost at two players disappears (0.472 for the curriculum and 0.475 for the hard-table network against the saver, against 0.452 for the control). Pooling the three seed sets (descriptively, since the development set suggested the hypothesis) the curriculum gains +0.043 [0.014, 0.071] at three players and +0.037 [0.012, 0.062] at four. We read this as a small gain at three and four players, about the size of the seed-to-seed variation, and a clear one from five players up.

Training on the official reward (exploratory, negative). Every learned agent above maximises the ±1 game outcome, which Section 5.3 shows is the secondary metric. If the eleven null results are a limit of the objective rather than of the method, replacing the reward by the official score should fix them: the difference between policies that are good at points and good at games is *when to release the joker*, a decision the network already sees (its move vector contains the jokers used and whether the move empties the hand) but never had a reason to use. We therefore retrained with reward points/60 (the liability of two jokers, which puts the reward on the ±1 scale the learning rate expects), from the same three clones, with the same recipe that is the paper's best (K=4 group advantages, 300 iterations of 24 games, learning rate 3×10^{-3}, $N = 2$). The control is the three group-advantage networks of Table 13, unchanged. Criteria were fixed before running: a gain of ≥ 3 points over the control against max-tiles, a non-negative score in points against max-tiles, and no regression below 0.45 in win score; development seeds 230,000+ (100 seeds, both seats, four opponents, 4,800 games), with test seeds 235,000+ reserved.

The reward changes behaviour in the predicted direction and does not change the outcome. The trained networks end a game holding 0.22 jokers against 0.28 for the control, go out with a joker in 58% of their wins against 69%, and pay −33.2 points per loss against −38.7: they learned to release the wildcard earlier, which is exactly what Section 5.3 shows is worth points. But the size is small. Against max-tiles they gain +1.7 [−0.7, +4.1] points over the control, short of the +3 we had fixed and with an interval containing zero, and they score −5.1 [−8.3, −2.0] points against max-tiles in absolute terms, so they remain clearly *below* it on the very metric they optimise. Win score is unharmed (0.530 [0.479, 0.581], the only criterion met). Pooled over the four opponents

the gain is $+3.3 \pm 1.6$ points with no change in win score ($+0.002 \pm 0.020$), and it is consistent across the three training seeds (-4.8, -5.0, -5.6 against max-tiles, against -7.8, -5.3, -7.4 for the control), so the effect is real but an order of magnitude smaller than the 5.6-point gap it would have to close. Training-time reward rose from $+0.08$ to $+0.18$, so the runs did learn. We did not spend the held-out seeds.

This closes the objection that the negative RL results are an artefact of the reward: with this architecture, this budget and this recipe, the network fails to beat the best heuristic under *both* scoring metrics, and the one behaviour it does acquire from the official reward (dumping the joker earlier) is a behaviour we can also write in one line as a heuristic, where it is worth about 4.5 points (Section 5.3). What limits these agents is not which number we reward. Section 5.14 removes the budget from the same list.

Counting hidden tiles (exploratory, negative). None of our strategies or features knew how many copies of each tile can still appear, although the table and one's own hand fix that count exactly (two copies per colour and number per 106-tile set, minus what is visible). We tried it in two ways, with the protocol fixed before running (120 seeds and both seats per cell, seeds 210,000+, N= 2 to 4, pairing as above). *Offensive:* `conta_rara` breaks ties of max-tiles by the pairs in the remaining hand that can still be completed, each weighted by the copies still hidden. *Defensive:* `defensiva_copia` holds back tiles whose last copies are in one's hand and that would extend a table combination, accepting fewer tiles played unless the move empties the hand; `defensiva_numero`, which holds back numbers whose group the opponents cannot complete, fires in 0.3% of decisions and was kept descriptive. None met the bar. At two players `conta_rara` scores 0.408 [0.346, 0.471] against the saver and 0.512 against the same-length-move strategy without counting; `defensiva_copia` differs from the saver by $+0.002$ [-0.061, 0.064] at three and $+0.007$ [-0.045, 0.060] at four players. We then added four counting features to the move vector (mean rarity of the tiles spent and of those kept, share of blocking tiles spent, number of blocking types held; $19 \rightarrow 23$) and retrained with the same recipe, three seeds and two regimes: twenty-three minus nineteen features, same regime and same hands, is -0.007 [-0.044, 0.030] at two players (two-player regime) and $+0.009$ [-0.012, 0.030] at four (curriculum). Cloning agreement stays at 0.82. We did not spend the held-out seeds. Figure 13 places this result beside the two other feature-set hypotheses: all three sit on zero. Counting is a real information source that the heuristics and the network did not turn into wins; with the games here, the joker rule still carries the advantage.

An external, independent opponent (exploratory). All matches so far pit our strategies against each other, so we also played them against code from outside this project: the open-source Rummikub solver of [25] (MIT-licensed, built on the integer-linear-programming formulation of [2]). Its default rule set matches ours exactly (thirteen numbers, two copies, four colours, two jokers, a thirty-point opening), confirming that our two-to-four-player engine is standard Rummikub. The solver maximises tiles placed on a given turn but does not decide when to draw or hold, so we wrapped it in the obvious greedy player: ask for the best placement, play it if it melds anything, otherwise draw. Against this external player our strategies dominate (Figure 14): the solver wins 0.040 [0.015, 0.064] of two-player games against the saver and never against max-tiles; in the mixed three- and four-player field it wins essentially nothing (0.001 and 0.003), against 0.56 and 0.40 for the saver and 0.57 and 0.43 for max-tiles. The solver draws about forty times and melds thirteen per game, where the saver melds twenty-one: optimal per-turn placement is not a policy, because it neither keeps jokers nor paces the draw. This measures the solver as a greedy player, not in its

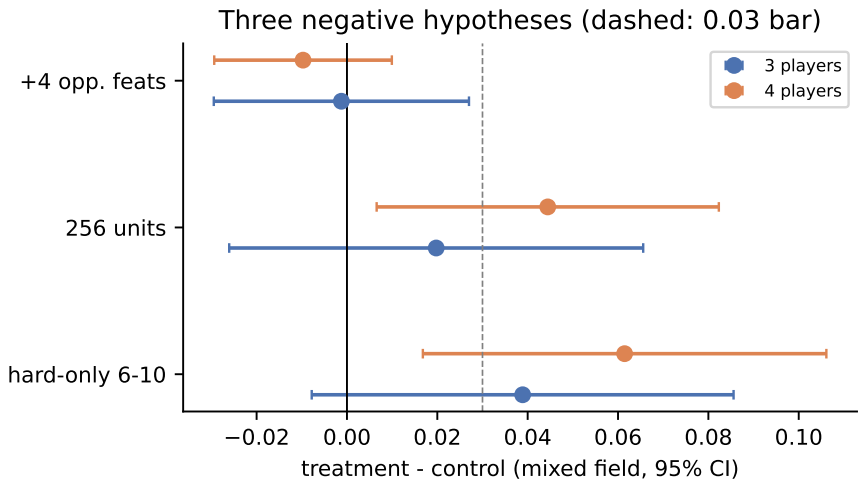


Figure 13: Three negative hypotheses, development seeds: treatment minus the two-player control in the mixed field (95% CI). None clears the pre-set 0.03 bar with a positive interval at both table sizes.

intended use (suggesting one turn's best move to a human); the gap is one of whole-game policy, not of rules. It does show our strategies are strong against an opponent we did not write.

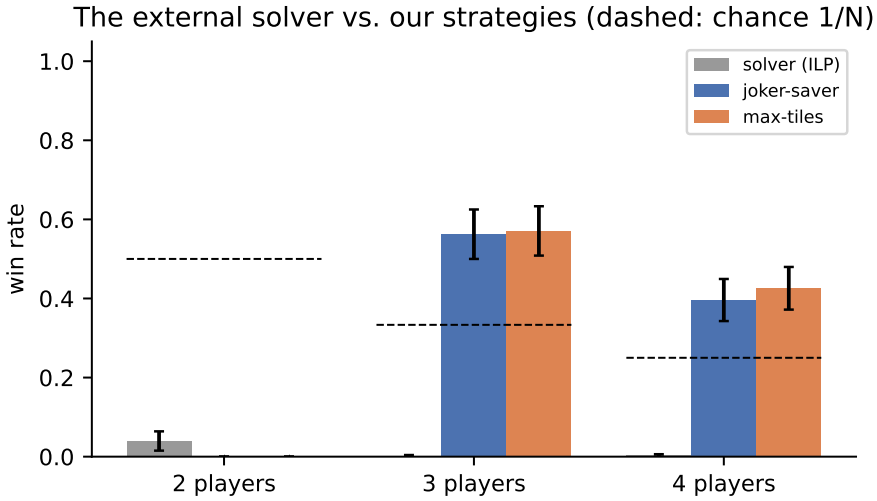


Figure 14: The external ILP solver, wrapped as a greedy player, against our two best heuristics (dashed: chance $1/N$). It wins almost nothing.

Do networks trained on harder tables beat the external player? The experiments above all measure our networks against our own heuristics; the right test of the "train harder, play easier" idea is against the independent player. We played six trained networks against the solver at two, three and four players (Figure 15) (the network as focus, $N-1$ solvers in the field, same held-out seeds). All of them dominate: the curriculum, hard-tables-only, 256-unit and nineteen-feature networks win essentially every game at two and three players and about 0.99 at four, where three solvers share the table. The one exception is the 256-unit network trained only at two players,

which drops to 0.86 at four, the one network that never saw a large field. The curriculum networks also beat the saver here: at four players the saver wins 0.848 [0.809, 0.887] against the solver, below the curriculum network's 0.988. This does not contradict the small or null gains against our own heuristics: against the strong saver everyone is close, while against the weak external player almost everyone saturates near one, so the ceiling hides most differences. What survives the ceiling is that training on larger tables helps against an external opponent on a large table, and that a network trained only at two players is the one that struggles there.

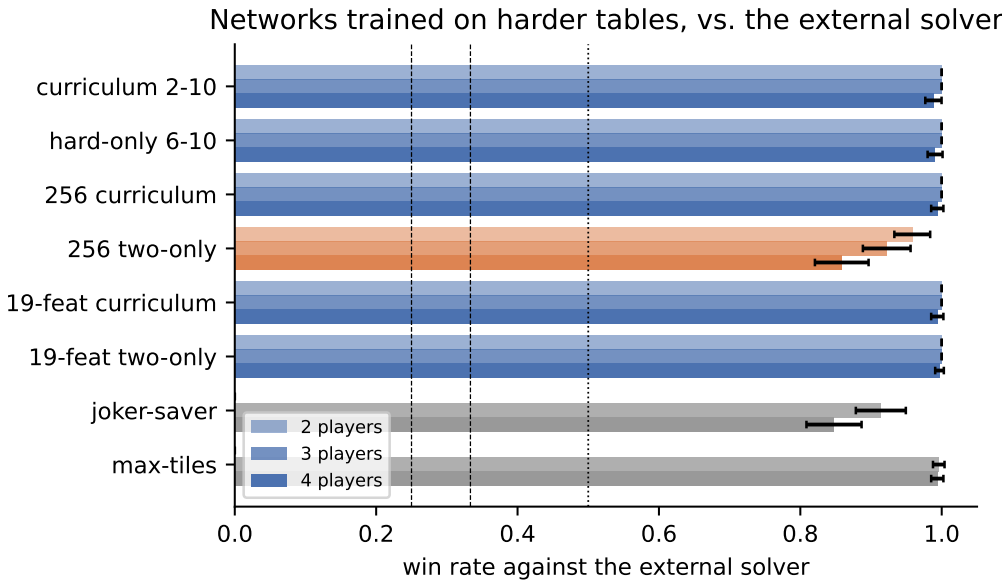


Figure 15: Networks trained on harder or larger settings, played against the external solver ($N-1$ copies in the field; shade = table size). All crush it except the 256-unit network trained only at two players; the joker-saver also trails at four players.

5.14 Scaling the training budget: the one variable we never moved

Every variant of Table 13 trained on 7,200 games (300 iterations of 24), and Section 7 declared that budget a limitation without ever measuring it. DouZero uses on the order of 10^9 games [6]; IRumAI uses 327.7 million PPO steps on a single GH200 [18]. We therefore repeated the paper's best recipe (clone of the joker-saver, group-relative advantages with K=4, official points as reward) changing *only* the budget: 4,000 iterations of 48 games, 192,000 games per training seed, three seeds, 576,000 games, 9.9 h of wall-clock on six CPU cores. Criteria were fixed before running: a gain of ≥ 4 points over the same recipe at the old budget against the joker-saver, a non-negative score in points, and no regression against max-tiles. Development seeds 250,000+, test seeds 255,000+ reserved.

Against the joker-saver the scaled agent scores +4.69 [+2.10, +7.28] official points (150 hands, 5,400 games), the first learned agent in this project whose interval excludes zero against the champion on the metric the rules use. The gain over the same recipe at 7,200 games is +2.83 [−0.84, +6.51], short of the +5 we had required. That interval crosses zero, so we re-measured it with the resolution it lacked: on 1,200 fresh hands (seeds 300,000+, 28,800 games, sized so the half-width is about one point) the gain is +3.57 [+2.24, +4.90]. The budget therefore does help, by a real amount, and

still does not reach the size we demanded. The third criterion failed because of how we wrote it: we required non-negative points against max-tiles, and the agent scores −2.79 [−4.27, −1.30] (−1.15 [−1.63, −0.67] on the 1,200 fresh hands); but both controls are also negative there (−4.75 and −8.86), because max-tiles is the strongest agent in points (Section 5.3). We had asked for a condition no learned agent in this paper ever satisfied, and we report it that way rather than rewrite the criterion after seeing the number. The held-out seeds were not spent.

The mechanism is the interesting part, and it inverts the usual story. The scaled agent *wins fewer hands* than its own control (0.390 against 0.487) and still scores more points, because it loses far more cheaply: −18.1 points per loss against −42.5. In defeat it ends with 2.78 tiles and 0.011 jokers in hand, against 4.31 tiles and 0.604 jokers for the control. It learned to release the joker and empty its hand *before* losing instead of chasing the win — a policy none of our heuristics implements, and the cleanest confirmation of this paper's central claim: the metric chosen determines the optimal policy.

The budget stopped binding early. The training return saturated near +10 points around iteration 450 of 4,000 and oscillated without trend afterwards; evaluating eleven checkpoints against the joker-saver gives +3.2 (iteration 250), +10.3 (1,000), +2.3 (2,250), +6.1 (4,000). The last 167,000 games bought nothing: 27× the games bought about 3.6 points and then stopped. Budget is therefore ruled out as the explanation for the eighteen null results, alongside capacity, state, hidden information, reward and opponent diversity.

Is the learned agent the best player? No — it trails the best by about a point. A duel against two opponents cannot support that claim, so we entered the three networks into the common-field tournament of Section 5.3: thirteen competitors, every pair, the same hands, both seats, 9,360 games, seeds 270,000+ (Table 16, Figure 16). The network ranks *first* in official points, $+16.96 \pm 1.38$ against max-tiles' $+15.60 \pm 1.44$. That ranking is a trap. The paired comparison on identical hands against the same nine common opponents gives −1.69 [−3.70, +0.31] — the interval includes zero and the sign is negative. In the head-to-head duel on 150 hands the network scores −0.60 [−2.75, +1.55] points and wins 0.492 of hands, which reads as a technical tie on both metrics. That reading was a failure of resolution. On 1,200 fresh hands, enough for a half-width near one point, the network scores −1.15 [−1.63, −0.67] points and wins 0.472 ± 0.012 of hands: a small loss that is real on both metrics. The rank-order lead is an artefact of each agent facing a slightly different field.

What does survive is narrower and still worth stating: the network beats the joker-saver in points head-to-head, +5.59 [+1.42, +9.75], while winning only 0.411 of the hands. It is the only learned agent here to manage that. But the best Rummikub player in this study remains a three-line heuristic, and in win rate the network is sixth of eleven. Reinforcement learning with 27× the budget closed most of the gap to max-tiles, to about one point per hand, but did not reach it.

5.15 Structured board representation: a negative about our compute, not about the game

With budget ruled out, the remaining untested input is the *shape* of the representation. Every feature vector in this paper is an aggregate scalar ($n/14$, sum/60, min(hands)/14), which destroys the two relations that define Rummikub: adjacent numbers form runs, and distinct colours of one number form groups. IRumAI's ablation is the most direct external evidence that this matters

Table 16: Common-field tournament including the scaled RL agent (three training seeds pooled as one competitor); 60 hands, both seats, 9,360 games, seeds 270,000+. The first-place finish in points does not survive pairing: see the text.

Agent	Official points	Win rate
points-net (RL)	$+16.96 \pm 1.38$	0.603 ± 0.028
joker-spender	$+15.60 \pm 1.44$	0.609 ± 0.023
max-tiles	$+15.60 \pm 1.44$	0.609 ± 0.023
saver-dump-near	$+15.45 \pm 1.63$	0.608 ± 0.032
max-points	$+14.50 \pm 1.58$	0.572 ± 0.029
saver-then-dump	$+9.94 \pm 2.44$	0.660 ± 0.031
joker-saver	$+7.92 \pm 2.77$	0.651 ± 0.031
min-play	$+7.56 \pm 1.69$	0.490 ± 0.033
random	$+4.27 \pm 1.76$	0.399 ± 0.035
cautious	-32.12 ± 2.65	0.133 ± 0.021
no-rearrange	-101.13 ± 2.85	0.010 ± 0.007

— replacing their dual-branch convolutional encoder with a width-matched MLP costs 8 to 11 percentage points, more than removing reward shaping (4.9) or the behaviour-cloning warm start, and the largest single contribution they report [18]. The geometry transfers exactly: their 4 suits $\times$ 13 ranks is our 4 colours $\times$ 13 numbers, so a 1×3 convolution along numbers sees runs and a 4×1 convolution along colours sees groups.

We implemented it: a $7 \times 4 \times 13$ tensor per move (remaining hand, tiles played, final table, unseen tiles, tiles that would complete a meld, tiles already paired, table covered by a valid meld), two convolutional branches with 16 filters each, max-pooling over colours, a 64-unit dense trunk, the original fifteen features kept as scalars so the comparison adds structure rather than trading information. The analytic gradient was verified against finite differences (12 components, $|\Delta| < 2 \times 10^{-3}$).

The plan's third criterion was a gate: cloning on the new architecture had to reach the 0.82 agreement with the joker-saver that *every* previous architecture reached (16, 64 and 256 units; 15, 19 and 23 features — all 0.82). It reached 0.706 on held-out decisions with the paper's 10,609 cloning samples, and 0.745 with three times the data. The diagnosis is variance, not bias: the convolutional network has 28,849 parameters against the MLP's 273 (106×), i.e. 2.7 parameters per cloning sample against 0.026, and tripling the data raised held-out agreement while closing the train–test gap from 0.097 to 0.043.

Extrapolating that curve puts the 0.82 gate somewhere between 10^5 and 10^6 cloning samples. Nine hundred cloning games already took 64 minutes; the comparison is with IRumAI's 10,000 demonstration games *plus* 327.7 million PPO steps at 2,400 steps/s on a GH200, against our six games per second on six CPU cores with `listar_acoes_validas` consuming 98% of the time. That is three to four orders of magnitude, and patience does not close it. We stopped at the gate and spent neither the development nor the held-out seeds.

This negative is categorically different from the others in this paper. Those are claims about Rummikub; this is a claim about our compute budget. **It does not license the statement that convolutional representations fail to help in Rummikub** — the architecture never left imitation, so the plan's question remains open. We report it because an unanswered question that consumed a working implementation is itself information for whoever tries next, and because the implementation is available and verified.

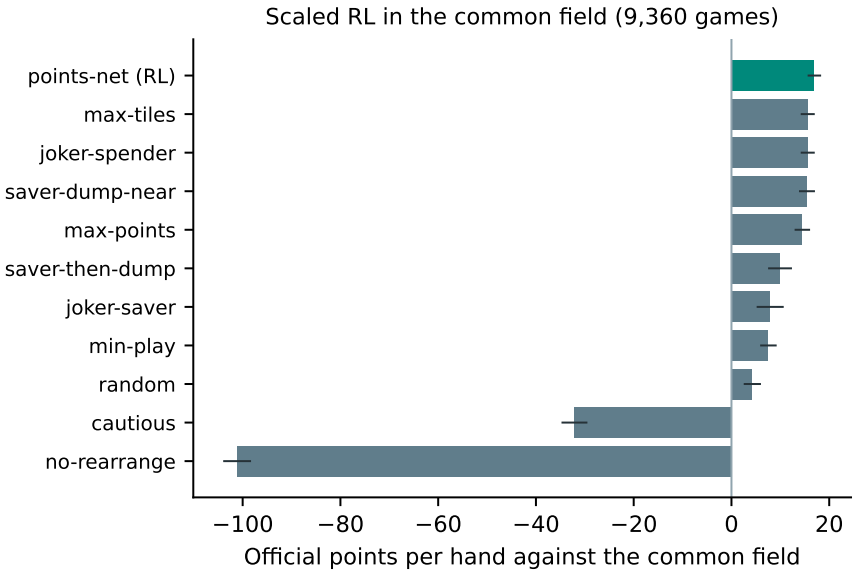


Figure 16: Official points per hand against the common field. The learned agent (dark) leads by +1.36 over max-tiles, but the paired difference on identical hands is −1.69 [−3.70, +0.31].

On joint-embedding predictive architectures. JEPA [20] is the obvious next idea and, we argue, the wrong one for this problem as posed. A JEPA predicts the *representation* of a masked or future observation instead of its raw content, with an EMA target encoder and asymmetric branches to avoid collapse; the gain is discarding unpredictable surface detail. That gain is defined against high-dimensional observations: I-JEPA and V-JEPA work on images and video, and the one study we found applying JEPA to reinforcement learning uses pixel CartPole, where the observation has 720,000 dimensions and the true state has four [21]. Our move tensor has 364 entries and is already the structured state; there is no surface detail to discard. The same study reports that JEPA without reinforcement gradients collapses to a constant embedding unless a variance regulariser is added, which is a sizeable extra failure mode for an architecture we cannot yet train to the imitation baseline. There is a genuinely interesting variant for our setting — latents that carry a *belief* over hidden continuations, since the opponent's hand is exactly such a hidden mode [22] — but the published demonstrations are oscillator and CartPole-scale, and our bottleneck would be identical to Section 5.15: parameters per sample, not ideas.

The real next step is the simulator, not the agent. `listar_acoes_validas` consumes 98% of every game, enumerating table rearrangements in pure Python; a game with rearrangement costs 1,006 ms against 167 ms without. Memoisation by table signature, dominance pruning, or the enumeration in C would multiply the effective budget of every future experiment, and is a precondition for any architecture with more than a few thousand parameters. We report this as the recommendation rather than as a result.

6 Discussion

The simulation supports a narrow, mechanistic answer to "what wins": *play the biggest legal move, but keep jokers until one of them lets you go out* — if winning means winning a single hand by game count. The three weaker explanations (soft preference, table recycling, timing alone) each fail, while the finishing-tile intervention both removes and reverses the effect. Practically, the common advice "save your jokers" is only half right twice over. First, unconditional saving is far worse than not saving (0.189): the condition "unless it wins the game" is the whole content of the rule. Second, the rule itself is an artefact of the measurement. Under the tile-point scoring printed in the rules, keeping a joker costs about six points a game against simply playing the most tiles, because the 30-point liability in the lost games outweighs the extra games won (Section 5.3); over sessions of ten hands, the format in which the game is actually played, the same policy wins the most hands and the fewest sessions (Section 5.4). The mechanism is the same object seen from two sides: a held wildcard is a bet that pays a win and charges 30 points when it fails, and which side dominates is decided by the scoring rule and the format, not by the game. Releasing the joker before the end improves points and sessions at once, so the frontier is not a strict trade-off everywhere. The same picture appears from the side of the tile: a joker, whether dealt or drawn, adds a clearly positive amount only for the joker-saver (Section 5.9); for a player that spends jokers freely it is worth about nothing. None of this hinges on how a stalled pool is scored (Section 5.5).

The measurement lesson generalises beyond this game. A per-hand win rate of 0.573, the kind of margin this literature routinely reports as an improvement, corresponded here to losing three sessions out of four, and the session ranking was the exact reverse of the per-hand win ranking on the five strategies measured both ways. Game count is the cheap, low-variance, conventional metric; points over a session is the one the rules define. On our menu they were anti-correlated, and no amount of extra seeds on the cheap metric would have revealed it.

The negative RL, Jev and search results are also informative. A tabular selector over a small state and a general-purpose language model choosing a strategy both gravitated to max-tiles-like behaviour; neither found the finishing-joker rule, which a human analyst reading the heuristic tournament did. Giving Jev the whole legal move list did not help: it was worse than the fixed heuristics, and a determinised search that rolls out with the joker-saver only matched it. A small policy network trained end to end on the move list, with no strategy menu, ended up at the same place: statistically level with max-tiles, below the joker-saver, and almost never going out with a joker. Against the joker-saver, every learned or searching agent except PIMC with $D = 2$ scores between 0.40 and 0.43; the exception among learned agents is the one started from a clone of the joker-saver (0.487 ± 0.045, Section 5.13), which ties with it, and also with group-relative advantages (0.495 ± 0.040). The finishing-joker rule is a delayed, rare event (it matters in the last turns of a game), which makes it hard for a policy-gradient signal at this budget to find; we did not test that explanation. Imitating the saver first restores part of the rule (more than half of the wins go out with a joker), and the group-relative advantage keeps it stable across training seeds, but none of the eighteen variants of Table 13 moves the score against max-tiles beyond 0.50–0.55, and against unseen opponents the network behaves like max-tiles (Table 14). Three of those variants are worth more than the rest, because they rule out the obvious diagnoses: a search given the *true* hidden hands does not improve, so hidden information is not the binding constraint; wider layers and richer features change nothing, so neither is capacity or state; and rewarding the official score makes the network release jokers earlier, exactly as the heuristics say it should, yet leaves it 5.1 points below max-tiles on the metric it optimises, so the reward is not it either. What remains is credit assignment for a rare event at the end of a hand, plus the strength of the baseline. The

contrast with IRumAI [18], which reaches 53.9% against a strong search baseline in Indian Rummy with the same clone-then-policy-gradient outline, suggests the difference lies in scale and observation design — meld-aware encodings, a convolutional policy, deadwood shaping — rather than in the recipe; we make no prediction about whether those ingredients would clear our baseline. A joker-aware state, an explanation of the rule in Jev's prompt, search with a different rollout policy, and reward that is dense only in the last turns of a hand are the natural next experiments.

7 Limitations

- **The objective and the format.** Every experiment other than Sections 5.3 and 5.4 scores a single hand as win, draw or loss, which is neither the metric printed in the rules nor the format in which the game is played. Those two sections cover the heuristics at two players only; the mechanism ablations, the league and the larger tables were all run on the ± 1 single-hand outcome. One set of networks was retrained with the official score as the reward (Section 5.13): it releases jokers earlier, as predicted, but stays below max-tiles in points. No agent was trained on a session return, and the session experiment uses $K = 10$ (with $K \in \{1, 5, 20\}$ in one cell), two players and one threshold for the adaptive rule. Sessions are also simulated as independent deals apart from the accumulated balance; we do not model opponents adapting across hands.
- **Menu.** "Best" means best of seven hand-written strategies, plus five exploratory ones in Section 5.8, three in Section 5.3 and one in Section 5.4; stronger policies may exist. Our only search agent (PIMC) used few worlds, the joker-saver as rollout policy and 30–60 seeds per comparison. The move network has a 16-unit hidden layer; hyper-parameters were not tuned, and only one value each of the learning rate, group size (4), shaping weight (0.5) and wider league was tried.
- **Joker identity on the table.** Officially a joker already melded must keep representing the same tile unless the player supplies that tile; our engine stores a joker as a value-free tile and validates only the final sets, so a rearrangement may silently change what a table joker represents. The "jokers frozen on table" ablation is stricter than the official rule and the default is more permissive, so the true rule lies between two conditions we measured, but was not itself measured. Since every main result concerns jokers, this is the fidelity gap we would close first.
- **Truncated actions.** Rearrangement plans are a sample of ≤ 128. Section 5.6 varies the cap for the seven heuristics only (ranking unchanged); Jev, PIMC and the move network keep the default cap, and even the largest cap is not a full search of the table.
- **Two players.** All learning experiments and the PIMC agent, and the mechanism ablations other than the three-player check, are for two players; Section 5.7 runs the heuristics and retrained learned agents at three and four, where the joker-saver's advantage shrinks (three players) or vanishes (four); the learned agents there have one training run per table size. The curriculum network of Section 5.13 was trained on tables of two to ten players but evaluated only at 2 to 6 and at 10 players; tables of seven, eight and nine were seen in training and never measured. All table sizes are simulated, with the same heuristic opponents; no human players.
- **Draws and the rule used.** The tournament, mechanism, Q-meta and Jev results use the draw-at-pool-exhaustion rule; only the tournament, the joker-saver ablations and Q-meta were

repeated under the official tie-break (Sections 5.5 and 5.12); Jev-meta was not. Jev-moves and PIMC were run only under the official rule.

- **Drawn jokers.** The effect of a drawn joker is an adjusted association: stratifying by the number of draws is needed because players who draw more are behind and also meet more jokers, but the number of draws is itself influenced by what the player holds, so the adjustment may remove part of a real effect. Only the initial deal is an exogenous assignment.
- **Mechanism evidence is within-simulator.** The interventions are on policies, not on game rules, and part of the effect (the joint before/after-opening effect) is unexplained. The mechanism experiments were designed after the ranking and were not pre-registered.
- **Statistics.** Normal-approximation intervals over seeds; tournament pairs are uncorrected; the Holm family covers only the mechanism table and the extended league.
- **RL and Jev.** Q-meta has a coarse state and 1,200 episodes per replica; the Jev-meta sample is 600 games, one prompt, one model version, and 49% of calls fell back. Jev-moves has 70 seeds per opponent, one prompt and one model version, and is exploratory; the from-scratch move network is one training run (no seed replicas), so its distance from max-tiles is not separated from run-to-run variation; the cloned variants have three training seeds, but all RL variants were evaluated on the same simulator, with several looks at the data (a first estimate of 0.605 against max-tiles did not replicate), and the p-values in Tables 12 and 13 are not corrected for multiplicity across rows. The three training seeds of each variant share deals, so pooled intervals are optimistic in the sense of [15]. Training the cloned networks with rearrangement on was run for one seed only.
- **Untested techniques.** We did not run an asymmetric critic trained with hidden information, a model of the opponent's hand inside the search, or search with a learned value function (Section 2); the tile-counting tests covered two hand-written blocking rules and a 16-unit network trained by REINFORCE, not other uses of the count; the negative results for learned agents are for the methods we ran, not for RL in general.

8 Conclusion

RQ1. In a paired tournament of seven strategies (3,360 games) the joker-saver ranks first (0.726 ± 0.027), with max-tiles and max-points statistically tied behind it; never rearranging and cautious play are much worse. **RQ1b.** That ranking is an artefact of the win/draw/loss metric. Scored by official tile points on the same 18,000 games, the joker-saver falls to sixth of ten, 5.6 [3.8, 7.4] points per game below max-tiles while still beating it in games won, because it ends its losses holding 0.80 jokers at 30 points each; Spearman between the two orderings is 0.64. **RQ1c.** Over sessions of ten hands with accumulated points, the format the rules define, the inversion is complete: the joker-saver wins the most hands of any strategy (5.4 of 10) and the fewest sessions (0.288 ± 0.030), losing 72.5% [66.8, 78.2] of sessions to max-tiles, and the effect grows with session length (0.558 at K=1, 0.754 at K=5 and K=20). The session ranking matches the per-hand points ranking (Spearman +0.90) and reverses the per-hand win ranking (-1.00). Releasing the joker when an opponent nears going out is first in both points and sessions won; switching to joker-saving when behind on the scoreboard loses (0.444), so the accumulated balance is information our rule turned into a deficit. **RQ2.** The joker-saver's advantage in hands won comes from holding a joker until it can complete the final move: soft penalties and table-recycling do not reproduce it, the effect arises mainly after the opening, and removing the finishing exception reverses it ($0.568 \to 0.189$); 71%

of its wins end with a joker. **RQ3.** A strategy-selecting Q-learner (6,000 training games), a Jev-based selector (600 games, US$1.23), Jev choosing among all legal moves (0.350 and 0.414 against max-tiles and the joker-saver), a small determinised search (0.433 against the joker-saver) and a policy network trained on the move list (0.403 against the joker-saver, 0.493 against max-tiles) did not beat the best fixed heuristic. Starting the network from a clone of the joker-saver and using group-relative advantages brings it level with both (0.550 ± 0.040 and 0.495 ± 0.040, exploratory, not significant after correction for the several variants tried); reward shaping, a wider training league, training with rearrangement and a residual network over the saver add nothing, Deep Monte Carlo stays below the saver (0.47) and PPO, level with it on development seeds (0.50), falls to 0.447 on held-out seeds, a search that sees the true hidden hands also stays at 0.508, and against twelve opponents (five unseen in training) it behaves like max-tiles. Rewarding the official score instead of the game outcome makes the network release jokers earlier (0.22 left in hand against 0.28, -33.2 points per loss against -38.7) but leaves it 5.1 points below max-tiles in points, so the negative result holds under both objectives. The joker-saver stays first under the official pool-exhaustion tie-break (0.755 ± 0.027) and the finishing-tile intervention keeps its effect (0.164). In a pooled field of 8,000 hands, a starting joker is worth at most about two points on average and a drawn joker adds $+0.119$ only to the joker-saver. These conclusions hold for this simulator, action sample and strategy menu; they do not establish a universally optimal Rummikub strategy. The broader methodological point does not depend on Rummikub: the conventional metric (hands won) and the metric the rules define (points over a session) were anti-correlated on our menu, so a 0.573 per-hand win rate meant losing three sessions in four.

Reproducibility. Code, per-game JSONL records and analysis scripts are in the project repository (`src/experimentos.py`, `src/estrategias/`, `src/jogo/pontuacao.py`, `src/coringas.py`, `src/analise*.py`, `resultados/`; run `python -m src.experimentos -help` for the subcommands, including `pontos` for Section 5.3). The Jev API key was read from the environment at run time and is not stored. Prompts, schema, cost and latency of both Jev agents are in Appendix A.

References

[1] J. N. van Rijn, F. W. Takes, J. K. Vis. The complexity of Rummikub problems. arXiv:1604.07553, 2016 (BNAIC 2015).

[2] D. den Hertog, P. B. Hulshof. Solving Rummikub problems by integer linear programming. *The Computer Journal*, 49(6):665–669, 2006. doi:10.1093/comjnl/bxl033.

[3] C. J. C. H. Watkins, P. Dayan. Technical note: Q-learning. *Machine Learning*, 8(3–4):279–292, 1992. doi:10.1023/A:1022676722315.

[4] M. van der Wekken. Algorithms for Rummikub puzzles. Bachelor thesis, LIACS, Leiden University, 2025.

[5] C. Kotnik, J. K. Kalita. The significance of temporal-difference learning in self-play training: TD-Rummy versus EVO-Rummy. *Proc. 20th Int. Conf. on Machine Learning (ICML)*, 369–375, 2003.

[6] D. Zha, J. Xie, W. Ma, S. Zhang, X. Lian, X. Hu, J. Liu. DouZero: Mastering DouDizhu with self-play deep reinforcement learning. *ICML*, 2021. arXiv:2106.06135.

[7] J. Li, S. Koyamada, Q. Ye, G. Liu, C. Wang, R. Yang, L. Zhao, T. Qin, T.-Y. Liu, H.-W. Hon. Suphx: Mastering Mahjong with deep reinforcement learning. arXiv:2003.13590, 2020.

[8] M. Balla, G. E. M. Long, J. Goodman, R. D. Gaina, D. Perez-Liebana. PyTAG: Tabletop games for multi-agent reinforcement learning. arXiv:2405.18123, 2024.

[9] M. Rudolph, N. Lichtle, S. Mohammadpour, A. Bayen, J. Z. Kolter, A. Zhang, G. Farina, E. Vinitsky, S. Sokota. Reevaluating policy gradient methods for imperfect-information games. arXiv:2502.08938, 2025.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[11] N. Brown, A. Bakhtin, A. Lerer, Q. Gong. Combining deep reinforcement learning and search for imperfect-information games. arXiv:2007.13544, 2020.

[12] J. Long, N. R. Sturtevant, M. Buro, T. Furtak. Understanding the success of perfect information Monte Carlo sampling in game tree search. *Proc. AAAI Conference on Artificial Intelligence*, 2010.

[13] P. I. Cowling, E. J. Powley, D. Whitehouse. Information set Monte Carlo tree search. *IEEE Trans. Computational Intelligence and AI in Games*, 4:120–143, 2012. doi:10.1109/TCIAIG.2012.2200894.

[14] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

[15] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, M. G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *NeurIPS*, 2021. arXiv:2108.13264.

[16] A. Y. Ng, D. Harada, S. Russell. Policy invariance under reward transformations: theory and application to reward shaping. *Proc. 16th Int. Conf. on Machine Learning (ICML)*, 278–287, 1999.

[17] I. Frank, D. Basin. Search in games with incomplete information: a case study using bridge card play. *Artificial Intelligence*, 100(1–2):87–123, 1998.

[18] V. Mohan et al. IRumAI: Reinforcement learning for Indian Rummy. arXiv:2606.21975, 2026.

[19] R. Feijen. Counting winning hands in Rummikub. Bachelor thesis, LIACS, Leiden University, 2023.

[20] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *CVPR*, 2023. arXiv:2301.08243.

[21] T. Zhang. JEPA for RL: investigating joint-embedding predictive architectures for reinforcement learning. arXiv:2504.16591, 2025.

[22] Anonymous. Predictive world models that imagine in belief space (UWM-JEPA). arXiv:2605.25313, 2026.

[23] J. McLeod. Rummikub. *Card Games website* (pagat.com), accessed October 2026. `https://www.pagat.com/rummy/rummikub.html`

[24] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6:65–70, 1979.

[25] M. J. Pieters. Rummikub solver library. `https://github.com/mjpieters/rummikub-solver`, 2025. MIT License.

A How Jev was used: prompts, schema, cost and latency

This appendix documents both Jev agents so that the experiments can be reproduced. Everything below is taken from the code (`src/estrategias/jev.py`, `src/estrategias/jev_jogadas.py`); the example payloads were produced by the same functions on a real mid-game position (seed 4000, 6 legal actions) without calling the API. No prompt was tuned on results: each agent used one prompt.

A.1 Interface

Jev is TypeSafe's System One model (`jev-1.13.0`, version pinned in the request). A request is a JSON object with three fields: `model`; `state`, a free-form JSON description of the situation; and `questions`, a map from a question name to a question. We only use questions of type `choice`, which carry an `instructions` string and a map `criteria` from option keys to option descriptions. The response contains, for each question, the selected key (`choice`) and a `confidence` in $[0, 1]$, plus `usage.input_tokens`, from which cost is computed (US$0.042 per million input tokens; output tokens are not billed). The call is a stateless HTTPS POST with a bearer key read from the environment. We set no sampling parameters, temperature or seed (none is part of our request), so Jev's answers need not be reproducible across runs. Transport errors and HTTP 429/529 are retried up to 6 times with exponential back-off (capped at 20 s, 60 s timeout); a persistent failure, an exhausted budget or an answer whose key is not among the options makes the agent fall back to max-tiles. The agents never produce moves themselves: the answer selects an option that the code has already validated, so no illegal move can be played.

A.2 Jev-meta (strategy selector)

State. Fixed text `game` ("Rummikub, 2 players, 106 tiles with 2 jokers; first to empty the hand wins."), the sorted hand, the table sets, whether each player has made the 30-point opening meld, the opponent's hand size and the pool size. Tiles are written in English ("blue 12", "joker").

Question. One `choice` question, *"Which option gives me the best chance of winning this game?"*, with seven options. Each option's description is a fixed sentence about the strategy followed by its concrete consequence *in the current position*, computed by running that strategy on the legal-action list (Table 17). The options are listed in a fixed order (they were not shuffled, so a position bias toward the first options was not controlled). If confidence is below 0.3 (fixed in advance), the agent plays max-tiles; 49% of calls were below this threshold. The prompt contains no argument for why one option should be preferred; in particular nothing explains the finishing-joker mechanism.

Table 17: Fixed part of the Jev-meta option descriptions. The code appends "Right now this would: *consequence*", e.g. "Play 1 tile(s) from the hand rearranging the table (tile sum 7, 0 joker(s) used)." or "Draw a tile from the pool and end the turn."

Key	Sentence
`max_pecas`	Play as many tiles as possible this turn.
`max_pontos`	Play the highest-value tiles this turn.
`poupar_coringa`	Play as many tiles as possible but keep jokers unless it wins the game.
`so_baixar`	Only lay down new sets; never touch the sets already on the table.
`minimo`	Play the smallest possible move and keep the rest of the hand.
`cauteloso`	Play only when a move uses at least 4 tiles (or wins); otherwise draw.
`comprar`	Draw a tile now.

```
{
 "model": "jev-1.13.0",
 "state": {
  "game": "Rummikub, 2 players, 106 tiles with 2 jokers; first to empty the hand wins.",
  "my_hand": ["blue 12", "blue 3", "blue 7", "green 4", "red 3", "yellow 13", "yellow 2",
              "yellow 6", "yellow 9"],
  "table_sets": [["yellow 11", "green 11", "red 11"], ["green 7", "green 8", "green 9",
                 "green 10"], ... 9 more sets ...],
  "i_have_made_my_initial_30_point_meld": true,
  "opponent_tiles_in_hand": 10,
  "opponent_has_made_initial_meld": true,
  "tiles_left_in_pool": 50 },
 "questions": {"estrategia": {
  "type": "choice",
  "instructions": "Which option gives me the best chance of winning this game?",
  "criteria": {
   "max_pecas": "Play as many tiles as possible this turn. Right now this would: Play 1 tile(s)
      from the hand rearranging the table (tile sum 7, 0 joker(s) used).",
   "so_baixar": "Only lay down new sets; never touch the sets already on the table. Right now
      this would: Draw a tile from the pool and end the turn.",
   ... 5 more options ... }}}}
```

Abridged example request (full request: 7 options, 11 table sets). Response format (the values here are illustrative, not a logged answer):

```
{"answers": {"estrategia": {"choice": "max_pecas", "confidence": 0.62}}, "usage": {"input_tokens":
1040}}
```

A.3 Jev-moves (concrete move selector)

Rules text. The state starts with a fixed rules paragraph:

```
Rummikub for 2 players. 106 tiles: numbers 1-13 in 4 colors (two copies of each) plus 2 jokers. Each
player starts with 14 tiles. A run is 3 or more consecutive numbers of one color. A group is 3 or 4
tiles with the same number and different colors. A joker stands for any tile. A player's first play
must be new sets worth at least 30 points, using only tiles from the hand. After that, a player may
add tiles to sets on the table and rearrange table sets, as long as every set on the table is valid
at the end of the turn. A turn is either playing at least one tile or drawing one tile. The first
player to empty the hand wins. If the pool is empty and both players pass in a row, the player with
the lower hand total wins (a joker counts as 30).
```

State and question. Besides the rules, the state lists the sorted hand, `hand_points_if_game_ends` (computed in code, jokers count 30), the table sets, the opening-meld flags, the opponent's hand size and the pool size. The question has one option per legal action of the turn (all of them: no top-K filter; a median of 30 and a maximum of 177 options were seen, below the 255 allowed), under keys `m001`, `m002`, ... assigned *after a seeded shuffle* so that option position carries no information. Each description states the tiles played from the hand, the number of tiles left, the table sets removed and added (runs written as "blue 5-6-7-8", groups as "group of 7s (green, red, yellow)", a joker as "joker") and, when applicable, that the move empties the hand. All sums and counts are computed in code, since Jev is weak at arithmetic. Turns with a single legal action (typically a forced draw) are not sent (32 of 59 decisions in the first game of the pilot).

```
{
 "model": "jev-1.13.0",
 "state": {
  "rules": "<text above>",
  "my_hand": ["blue 12", "blue 3", "blue 7", "green 4", "red 3", "yellow 13", "yellow 2",
              "yellow 6", "yellow 9"],
  "hand_points_if_game_ends": 59,
  "table_sets": [ ... 11 sets, as above ... ],
  "i_have_made_my_initial_30_point_meld": true,
  "opponent_tiles_in_hand": 10,
  "opponent_has_made_initial_meld": true,
  "tiles_left_in_pool": 50 },
 "questions": {"jogada": {
  "type": "choice",
  "instructions": "Each candidate is one complete turn I can play now. Which candidate gives
      me the best chance of winning this game?",
  "criteria": {
   "m001": "Play 1 tile(s) (tile numbers sum to 7, 0 joker(s) used); tiles from hand: blue 7;
      8 hand tiles left; table sets removed: blue 5-6-7-8; group of 7s (green, red, yellow);
      table sets added: blue 5-6-7-8; group of 7s (blue, green, red, yellow).",
   "m003": "Draw a tile from the pool and end the turn (hand grows to 10 tiles).",
   "m004": "Play 1 tile(s) (tile numbers sum to 7, 0 joker(s) used); tiles from hand: blue 7;
      8 hand tiles left; table sets removed: group of 7s (green, red, yellow); table sets
      added: group of 7s (blue, green, red, yellow).",
   ... 3 more candidates ... }}}}
```

Abridged example request (full request: 6 candidates, 11 table sets). The confidence threshold is 0: any valid key is played; an invalid key or transport failure falls back to max-tiles (7 failures in 280 games).

A.4 Cost and latency

Token counts and costs come from the `usage` field recorded by the experiments; wall-clock latency was *not* recorded during the experiments and was measured afterwards in a pilot of 62 Jev-meta and 27 Jev-moves calls (two games per agent; serial calls from one WSL2 machine, so the figure includes the network round trip but no queueing). Treat latency as indicative: one machine, one session, no confidence interval. Table 18 collects both agents. The wall-clock estimate per game multiplies calls per game by mean latency; it is a lower bound for a serial run (a retry can add up to 20 s) and the experiments ran several games in parallel. The total spend on Jev for all experiments, including pilots and the latency pilot (US$0.005), was about US$1.75, under the US$2 cap. Jev-moves is cheaper per game than Jev-meta only because it is queried on fewer decisions (forced draws are skipped); per call it costs about twice as much because the rules and the candidate list are longer.

Table 18: Jev usage. Cost is input tokens × US$0.042/M. Latency is per API call (pilot, $n = 62$ and 27).

	Jev-meta	Jev-moves
Games	600	280 (+8 pilot)
API calls	28,090	5,980 (+207)
Calls per game	46.8	21.4
Input tokens per call	≈1,040	≈1,980
Input tokens in total	29.2 M	11.8 M (+0.4 M)
Cost per call	US$0.000044	US$0.000083
Cost per game	US$0.0020	US$0.0018
Total cost	US$1.23	US$0.50 (+0.02)
Fallbacks / transport failures	13,853 / 7	7 / 7
Latency per call, mean (median, p95)	0.38 s (0.37, 0.46)	0.40 s (0.36, 0.39)
Maximum latency seen	0.54 s	1.37 s
Estimated API wait per game	≈18 s	≈9 s